Ensemble Learning

Thomas Schwarz, SJ

Ensemble Methods

If you ask a large enough group of diverse and independent people to make a prediction or estimate a probability, the average of those answers will cancel out errors in individual estimation. Each person's guess, you might say, has two components: information and errors. Subtract the errors, and you're left with the information.

James Surowiecki, in The Wisdom of Crowds

Ensemble Methods

- Before ANN, winners in Kaggle competitions were often
  - XGBoost
  - Random Forest
  - Gradient Boosting

Regression Example

- Friedman Set to test regression:
  - At least 5 explanatory variables
  - Create dependent variable

$$y = 10\sin(\pi x_1 x_2) + 20(x_3 - \frac{1}{2})^2 + 10x_4 + 5x_5 + \mathcal{N}(0,\sigma)$$

Regression Example

- Implemented in sklearn

```
from sklearn.datasets import make_friedman1

X, y = make_friedman1(n_samples=500,
                      n_features=15,
                      noise=0.3,
                      random_state=SECRET)
```

- Use a DecisionTreeRegressor

```
model = DecisionTreeRegressor()
```

  - A decision tree with output nodes that give constant values

Regression Example

- Break into training and test set

```
from sklearn.model_selection import ShuffleSplit,
validation_curve

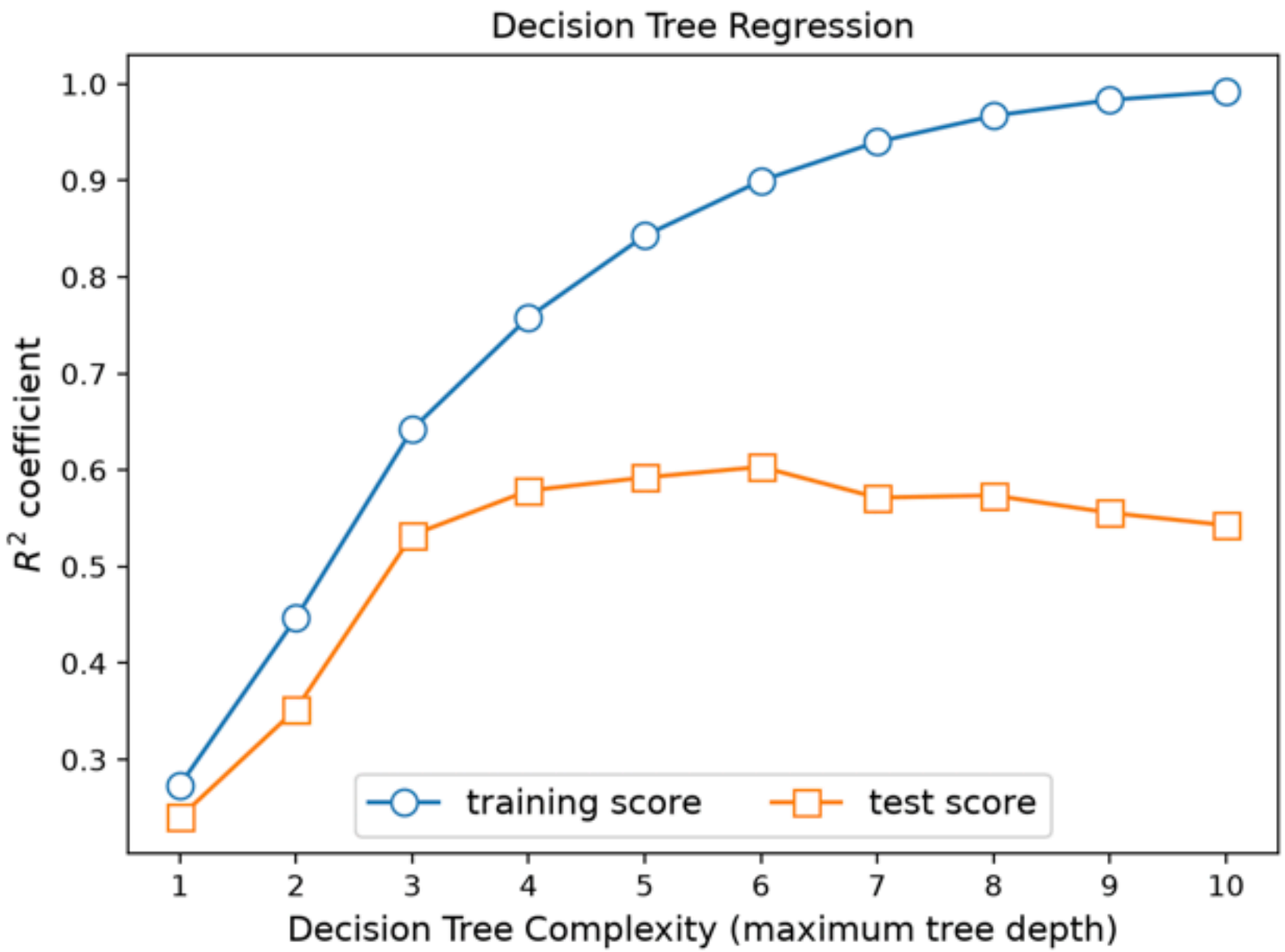
subsets = ShuffleSplit(n_splits=5, test_size=0.25,
                       random_state=SECRET)
```

- Create a validation curve using average of five random splits

```
trn_scores, tst_scores = validation_curve(model, X, y,
            param_name='max_depth',
            param_range=range(1, 11),
            cv=subsets,
            scoring='r2')
mean_train_score = np.mean(trn_scores, axis=1)
mean_test_score = np.mean(tst_scores, axis=1)
```

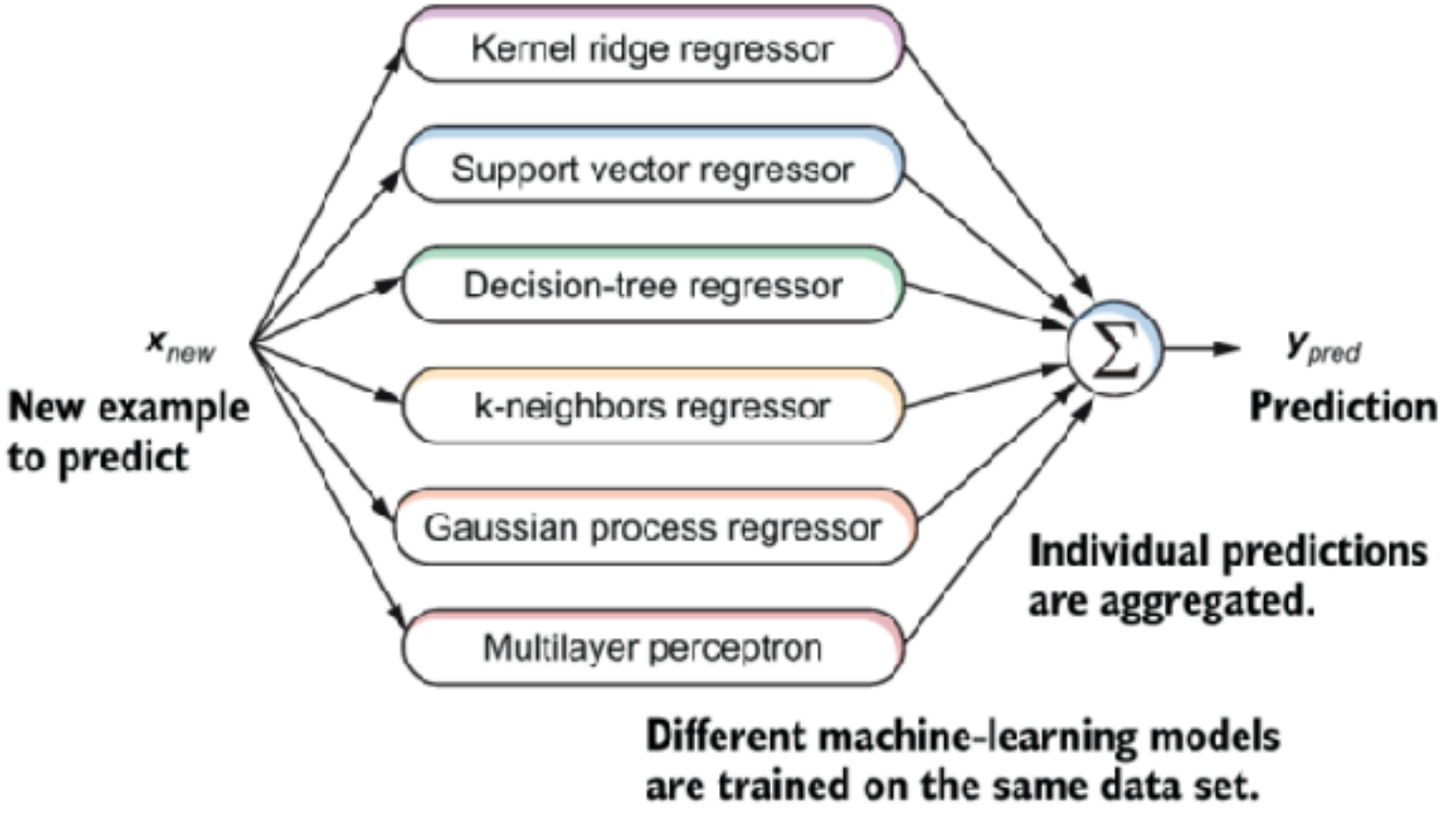
Regression Example

- Typical result for overfitting



Ensemble Learning

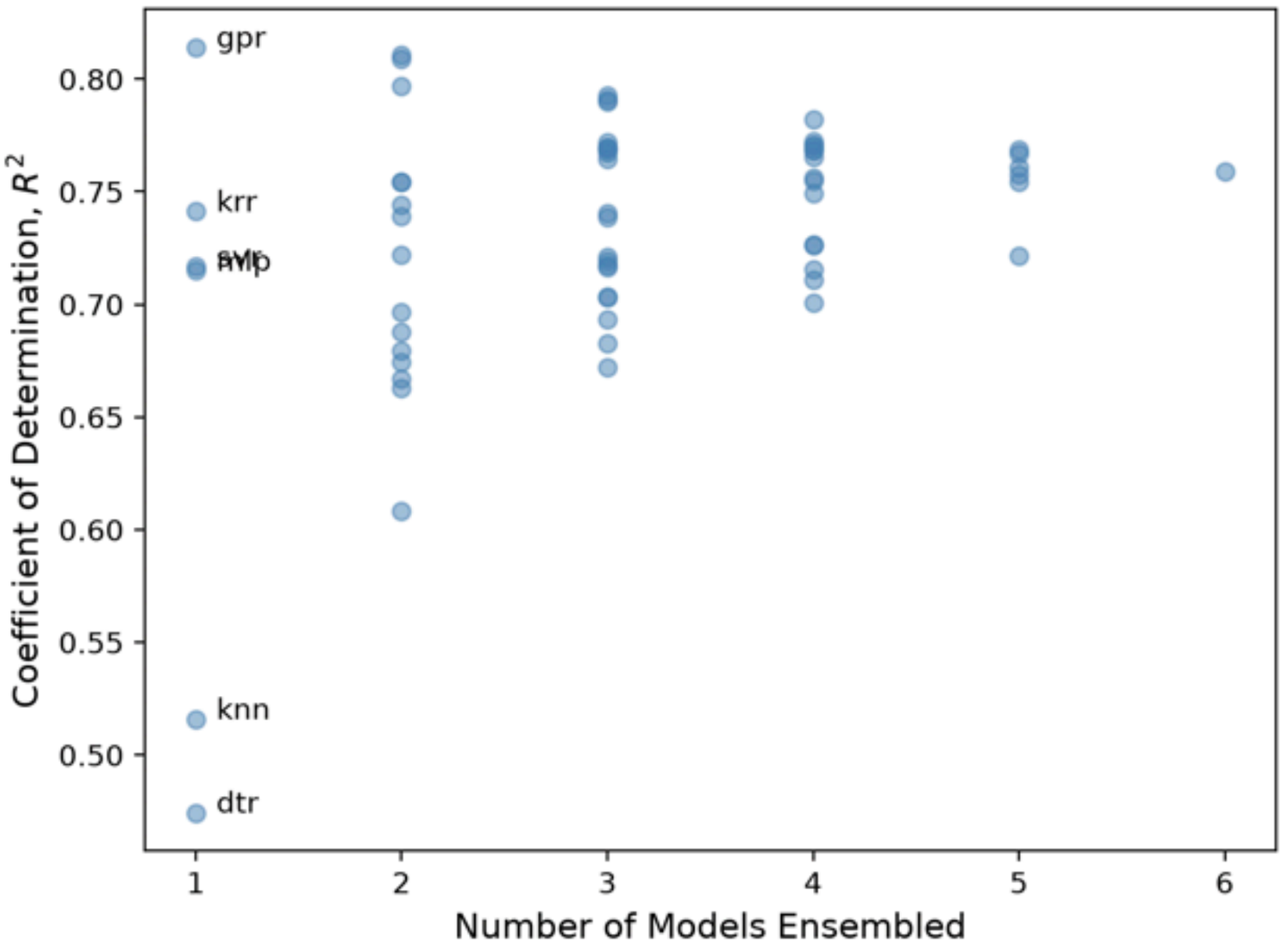
- Use different machine learning models on the same training set
- Aggregate the results



Ensemble Learning

- We look at all possible combinations of models for aggregation and see how they work
  - The results vary widely based on the set
  - As we use more methods, the results cluster closer together

Ensemble Learning



Ensemble Learning

- Types of ensemble learning
- **Parallel Homogeneous Ensemble**
    - Strong learners with same learning algorithm
    - But select random training sets or feature selection

Ensemble Learning

- **Parallel Heterogeneous Ensembles**
  - Strong learners
  - Use different training algorithms / parameter settings

Ensemble Learning

- **Sequential adaptive boosting**
  - Use weak learners trained in a stage-wise, sequential manner
  - Each successive model is trained to fix the mistakes of the previous one
  - Examples: AdaBoost, LogitBoost

Ensemble Learning

- **Gradient boosting ensembles**
  - extend adaptive boosting
  - mimic gradient descent
    - Example: XGBoost

Bagging and Random Forests

- Bagging,(bootstrap aggregating)
  - Leo Breiman 1996
  - Training with bootstrap sampling (sampling with replacement) generates different training sets out of a single training set
  - For classification tasks, we can combine individual predictions using majority voting.
  - For regression tasks, we can combine individual predictions using simple averaging.

Bagging and Random Forests

- Example: Use the previous example

```
SECRET = 15121957
MODEL_NUMBER = 20
RNG = np.random.default_rng(SECRET)
```

```
X, y = make_friedman1(n_samples=500,
                      n_features=15,
                      noise=0.3,
                      random_state=SECRET)

Xtrn, Xtst, ytrn, ytst = train_test_split(X, y, test_size=0.25)

models = [DecisionTreeRegressor(max_depth=5,
                                random_state=SECRET+1)
          for i in range(MODEL_NUMBER)]
```

Bagging and Random Forests

- For recursion, we use the mean

```
y_pred = np.mean(np.array([model.predict(Xtst) for model
                           in models]),axis=0)
```

```
Bagging 0.7766666822997983
Model 0: 0.3551139784426697
Model 1: 0.28407483807402445
Model 2: 0.3641395993397015
Model 3: 0.3780386606814007
Model 4: 0.2506630527052323 7
Model 5: 0.19180284400474212
Model 6: 0.5170777233678773
Model 7: 0.420214602686838 2
Model 8: 0.4008169547294286
Model 9: 0.3616029490867941 3
Model 10: 0.4054902366230112
Model 11: 0.3611368641982853
Model 12: 0.43454047452204947
Model 13: 0.3227115006809713
Model 14: 0.4209546838413496
Model 15: 0.374879835162807
Model 16: 0.427812770791861
Model 17: 0.3520936916192413
Model 18: 0.3642315183851253 5
Model 19: 0.3105356625079584
```

Bagging and Random Forests

- Classification example
  - For synthetic examples, we can use moons

```
SECRET = 12151962
RNG = np.random.RandomState(seed=SECRET)
X, y = make_moons(n_samples=300, noise=.25, random_state=RNG)
Xtrn, Xtst, ytrn, ytst = train_test_split(X, y,
                                          test_size=0.33, random_state=RNG)
```

Bagging and Random Forests

- We create a bunch of trees and train them on samples from the training set

```
def bagging_fit(X, y, n_models, max_depth=5, max_samples=150):
    n_examples = len(y)
    models = [DecisionTreeClassifier(max_depth=max_depth)
                  for _ in range(n_models)]
    for tree in models:
        bag = np.random.choice(n_examples,
                               max_samples, replace=True)
        tree.fit(X[bag, :], y[bag])
    return models
```

Bagging and Random Forests

- We use majority vote to determine the prediction

```
def bagging_predict(X, models):
    all_predictions = np.array([tree.predict(X) for
                                tree in models])
ypred, _ = mode(all_predictions, axis=0, keepdims=False)
return np.squeeze(ypred)
```

Bagging and Random Forests

- We use accuracy to measure success

```
models = bagging_fit(Xtrn, ytrn, n_models=50,
                     max_depth=10, max_samples=300)
ypred = bagging_predict(Xtst, models)
print(f"Bagging {accuracy_score(ytst, ypred):8.6f}")

tree = DecisionTreeClassifier(max_depth=10)
tree.fit(Xtrn, ytrn)
ypred_single = tree.predict(Xtst)
print(f"Single Tree {accuracy_score(ytst, ypred_single):8.6f}")
```

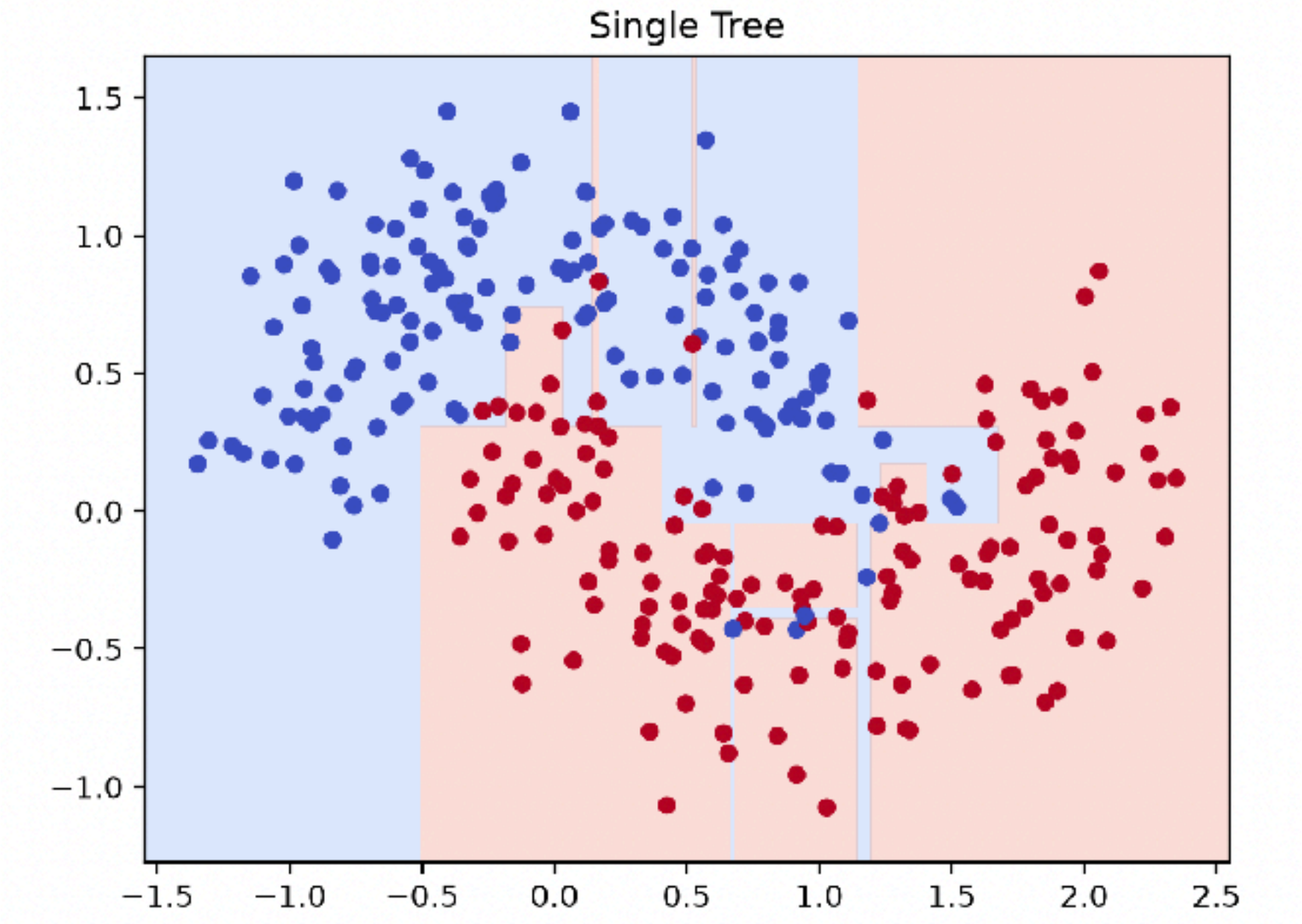
Bagging 0.959596

Single Tree 0.888889

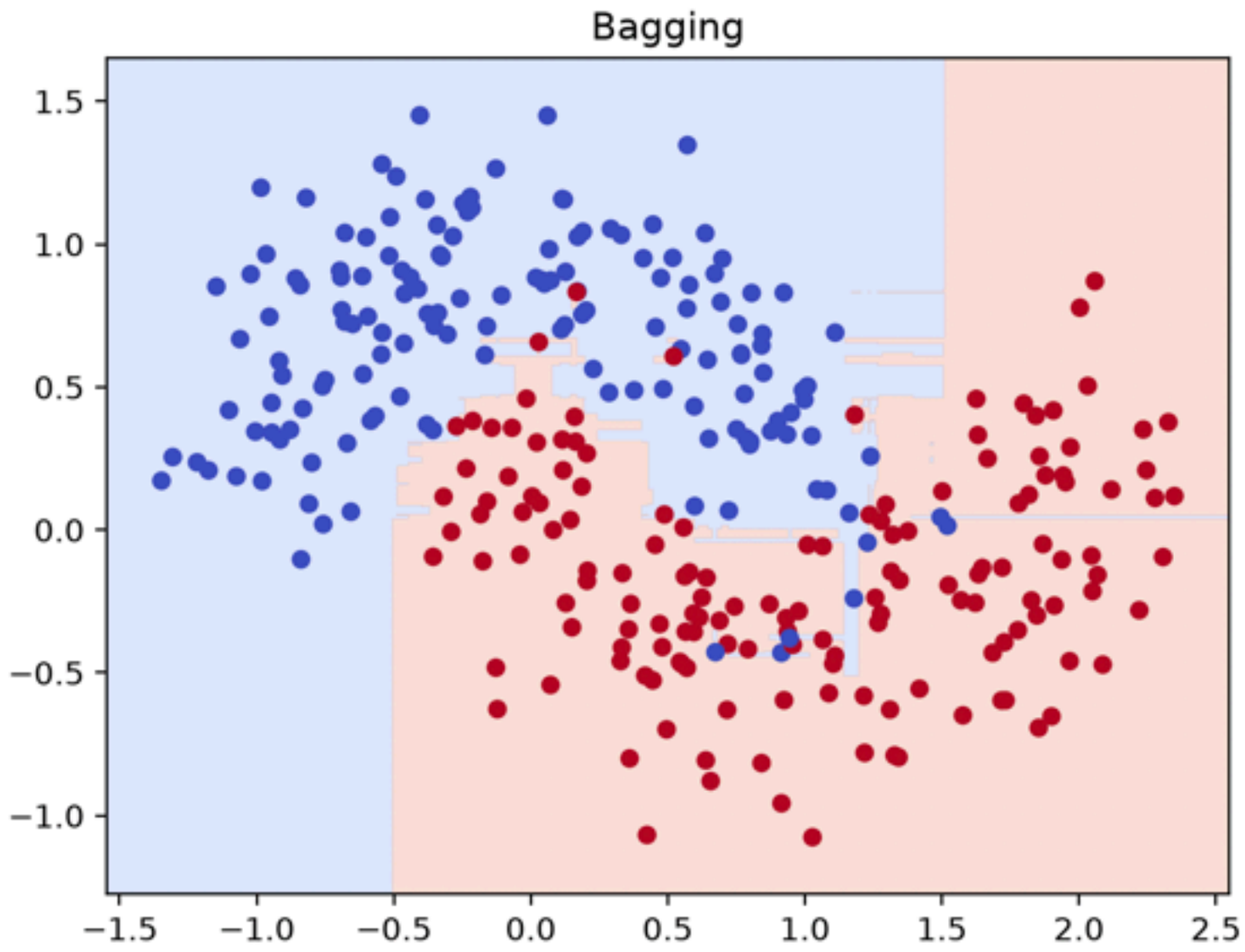
Bagging and Random Forests

- We use contour plots to show how little squares are predicted

Bagging and Random Forests



Bagging and Random Forests



Bagging and Random Forests

- Difference:
  - Single tree makes weird exceptions
  - Bagging eliminates those to a large extent

Bagging and Random Forests

- Bagging is implemented in sklearn
- OOB are out-of-bag data. Can be used for estimating generalization error

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

base_estimator = DecisionTreeClassifier(max_depth=10)
bag_ens = BaggingClassifier(base_estimator=base_estimator,
                            n_estimators=500,
                            max_samples=100,
                            oob_score=True,
                            random_state=rng)
bag_ens.fit(Xtrn, ytrn)
ypred = bag_ens.predict(Xtst)
```

Bagging and Random Forests

- Random forest
  - Use bootstrap sampling
  - Use randomized decision trees
    - Based on using a random subset of features
      - This incidentally allows determining importance of a feature

Bagging and Random Forests

- Scikit-learn implements Random Forests
  - Can use Out-Of-Bag error to estimate generalization error
  - Can determine the importance of features

Bagging and Random Forests

- Feature Engineering
    - Removing least important features can reduce generalization error

Bagging and Random Forests

- Small synthetic example:

```
rng = np.random.RandomState(seed=12151963)
X, y = make_moons(n_samples=500, noise=.25, random_state=rng)
Xtrn, Xtst, ytrn, ytst = train_test_split(X, y,
                                          test_size=0.33, random_state=rng)
rf_ens = RandomForestClassifier(n_estimators=500, max_depth=10,
                                oob_score=True,
                                n_jobs=-1,
                                random_state=rng)
rf_ens.fit(Xtrn, ytrn)
ypred = rf_ens.predict(Xtst)
```

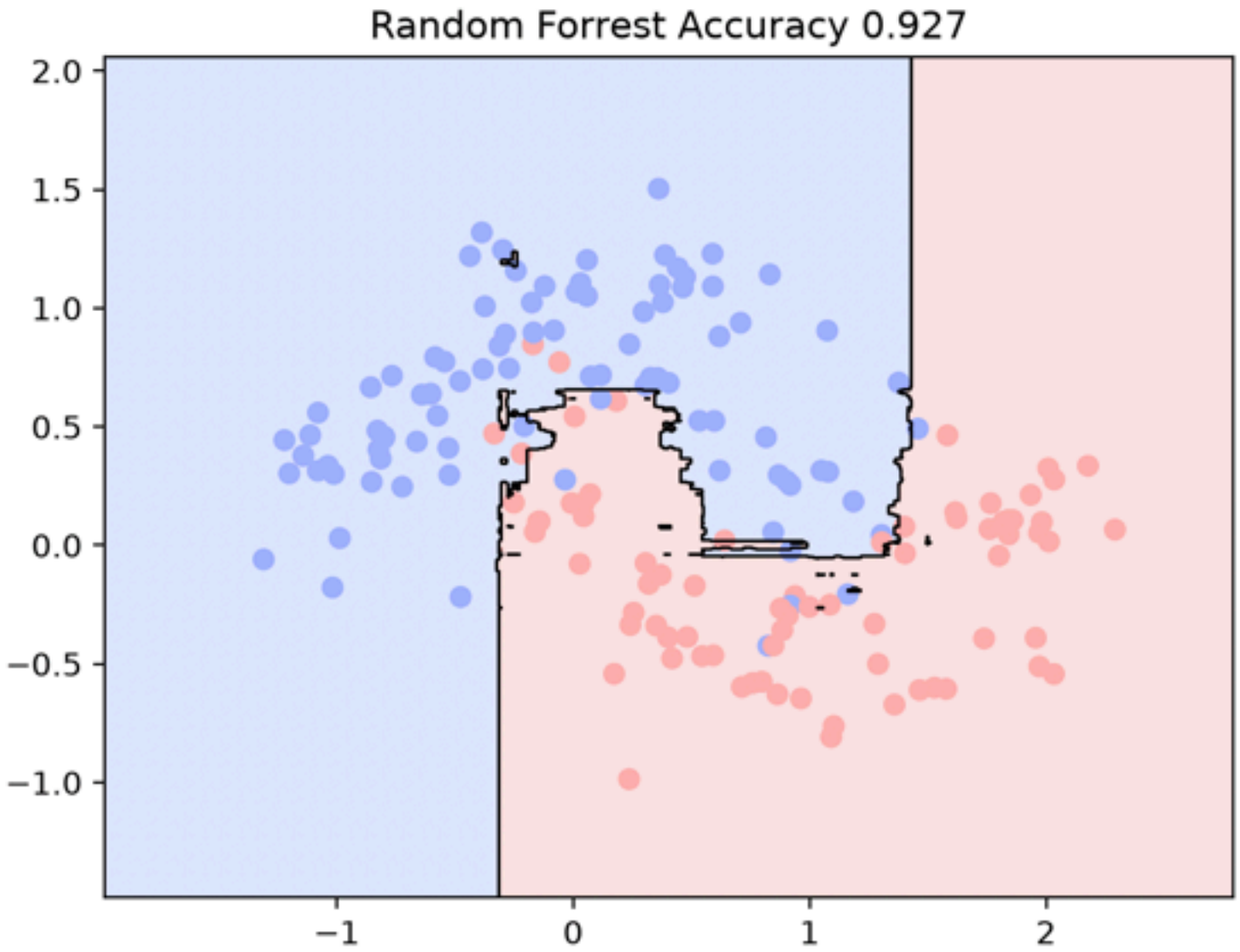
Bagging and Random Forests

- Feature importance

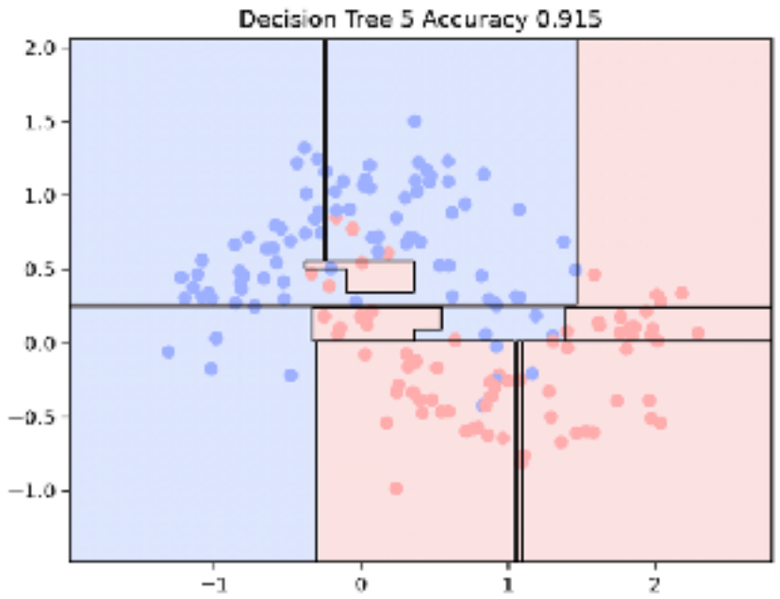
```
for i, score in enumerate(rf_ens.feature_importances_):
print('Feature x{0}: {1:6.5f}'.format(i, score))
```

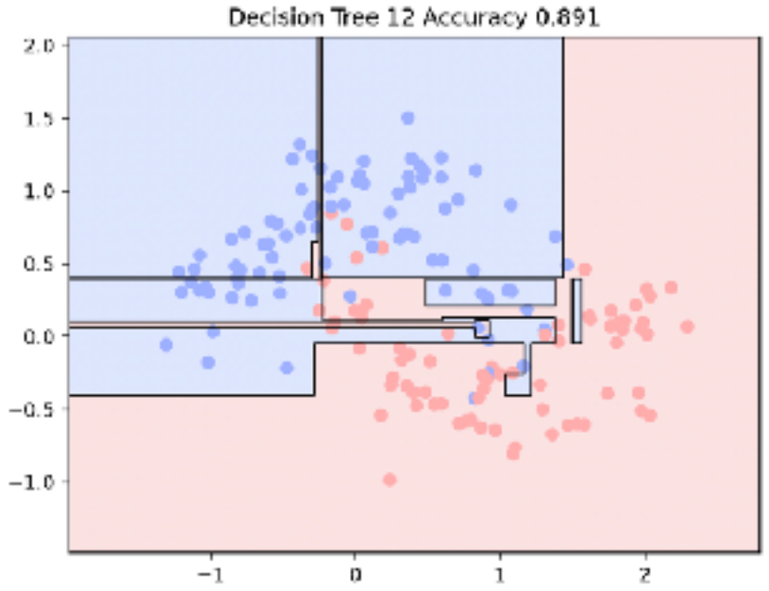
```
Python/ensemble5.py
Feature x0: 0.44758
Feature x1: 0.55242
```

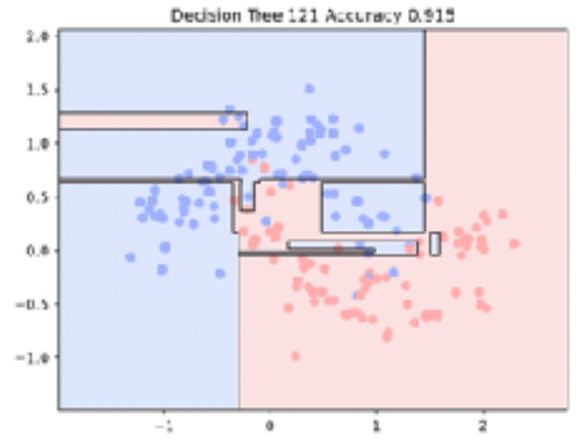
Bagging and Random Forests

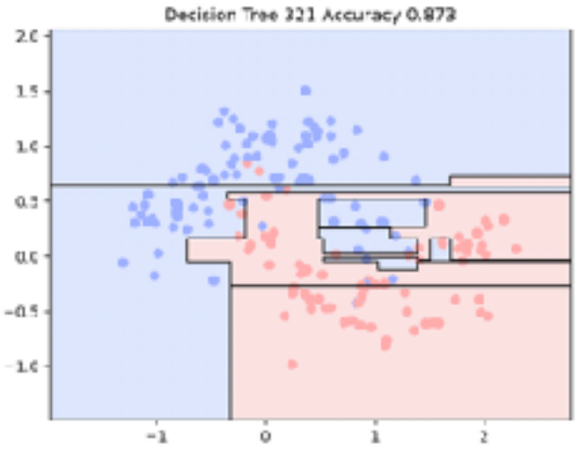


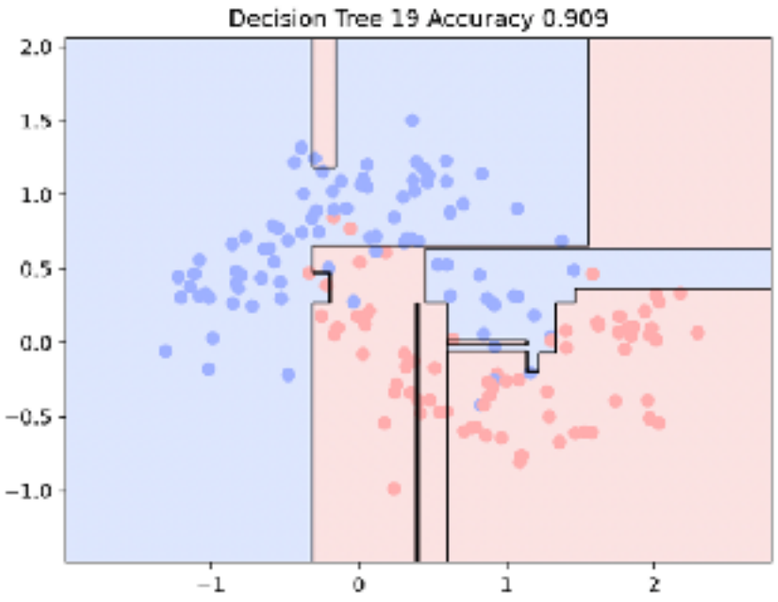
Bagging and Random Forests

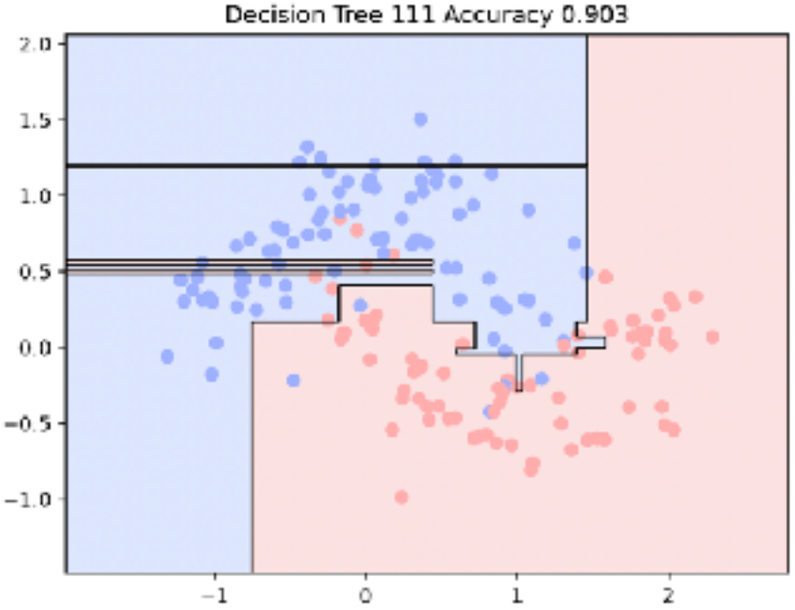


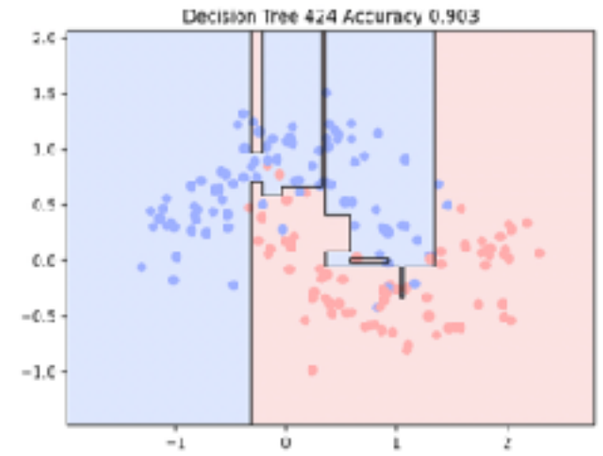












Bagging and Random Forests

- Feature selection example:
  - Create a random forest on the MNIST data set

```
forest_clf = RandomForestClassifier(n_estimators=500,
                                    random_state=SECRET)
forest_clf.fit(X, y)
```

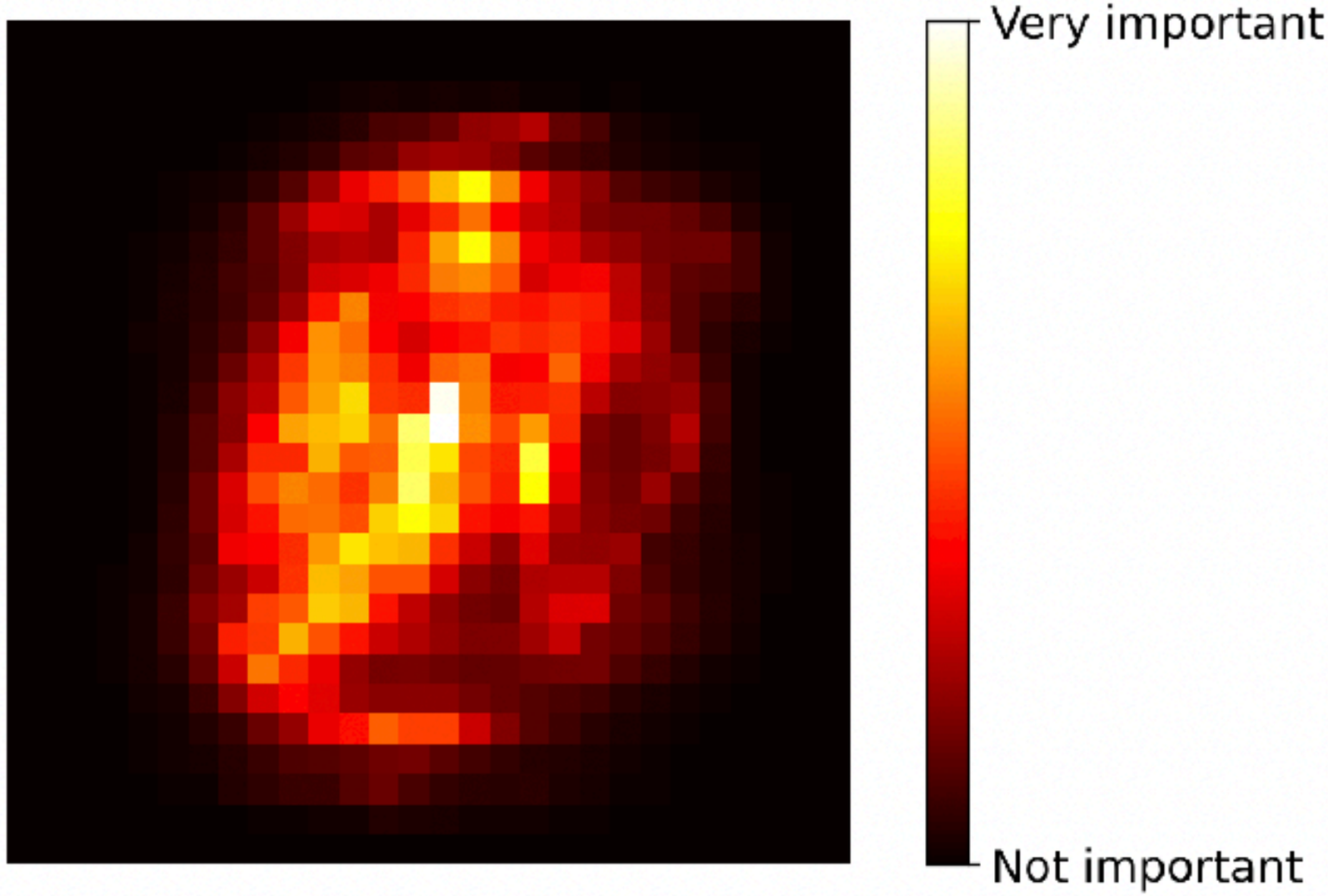
Bagging and Random Forests

- Display feature.importances_ in a heatmap

```
heatmap_image = forest_clf.feature_importances_.reshape(28, 28)
plt.imshow(heatmap_image, cmap="hot")
```

Bagging and Random Forests

- Feature Selection



Other Homogeneous Parallel Ensembles

- Pasting
  - If data set is large, use sampling without replacement instead of bagging
- Random subspaces
  - Select random features for the components
- Extremely randomized trees
  - Use a random splitting threshold for the component trees
    - `ensemble.ExtraTreesClassifier`

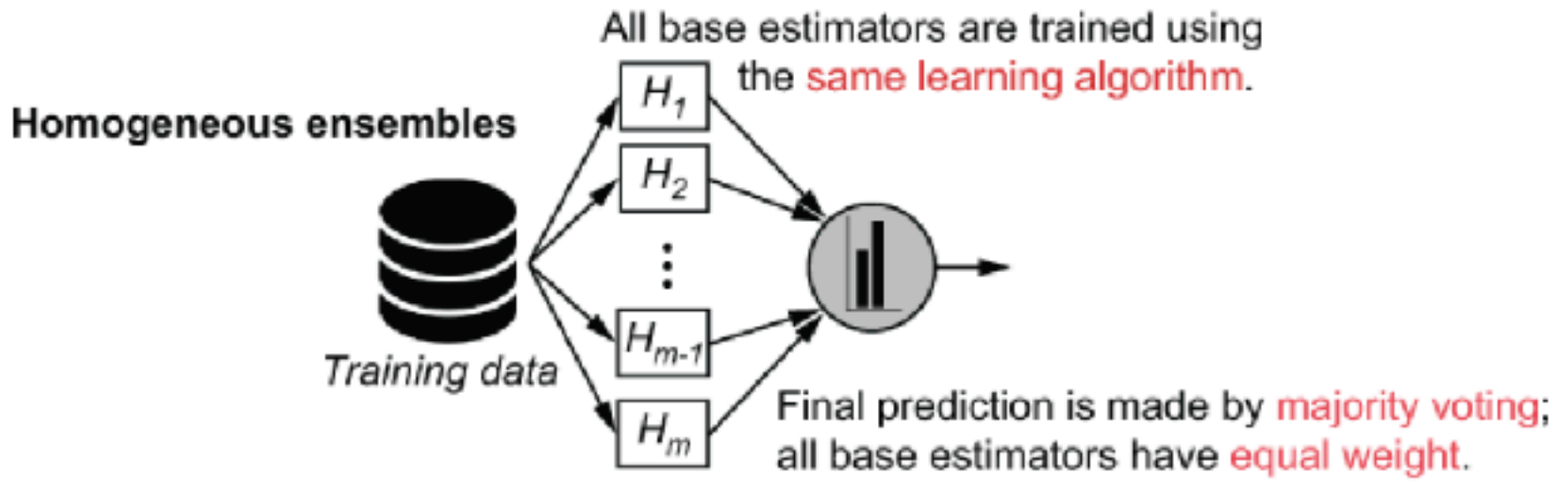


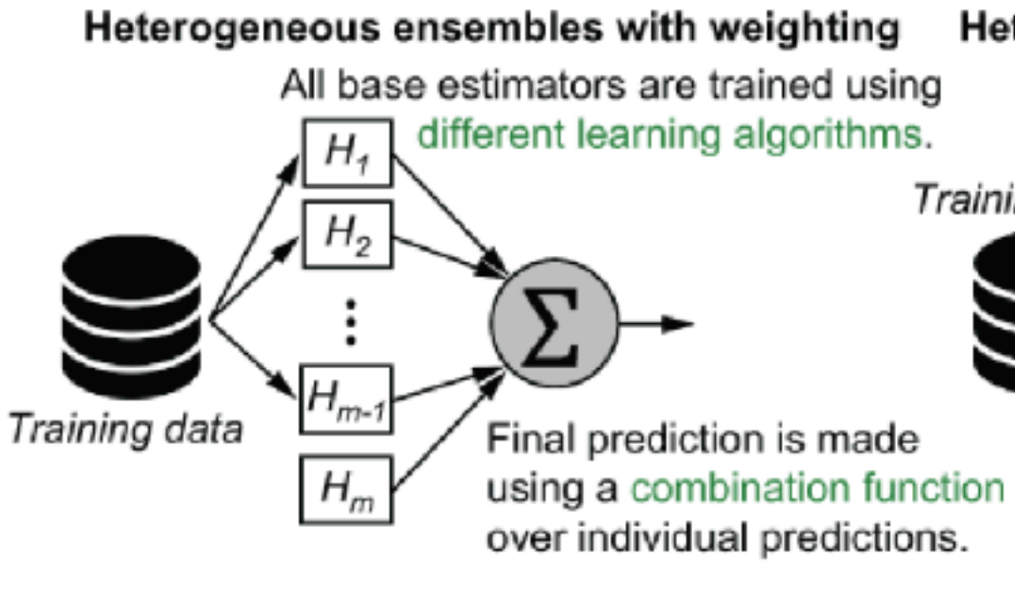
Heterogeneous Parallel Ensembles

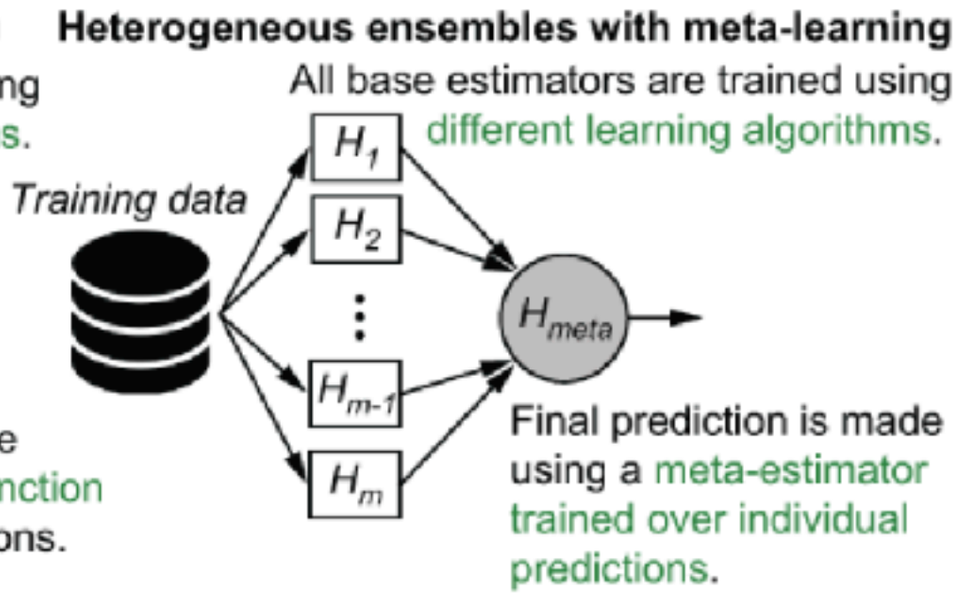
Heterogeneous Parallel Ensembles

- Create various strong estimators using different models
- For each new point, combine the predictions:
  - Use majority vote
  - Use weighted vote
- Use a metalearner
  - Trained on the outputs of the individual components

Heterogeneous Parallel Ensembles







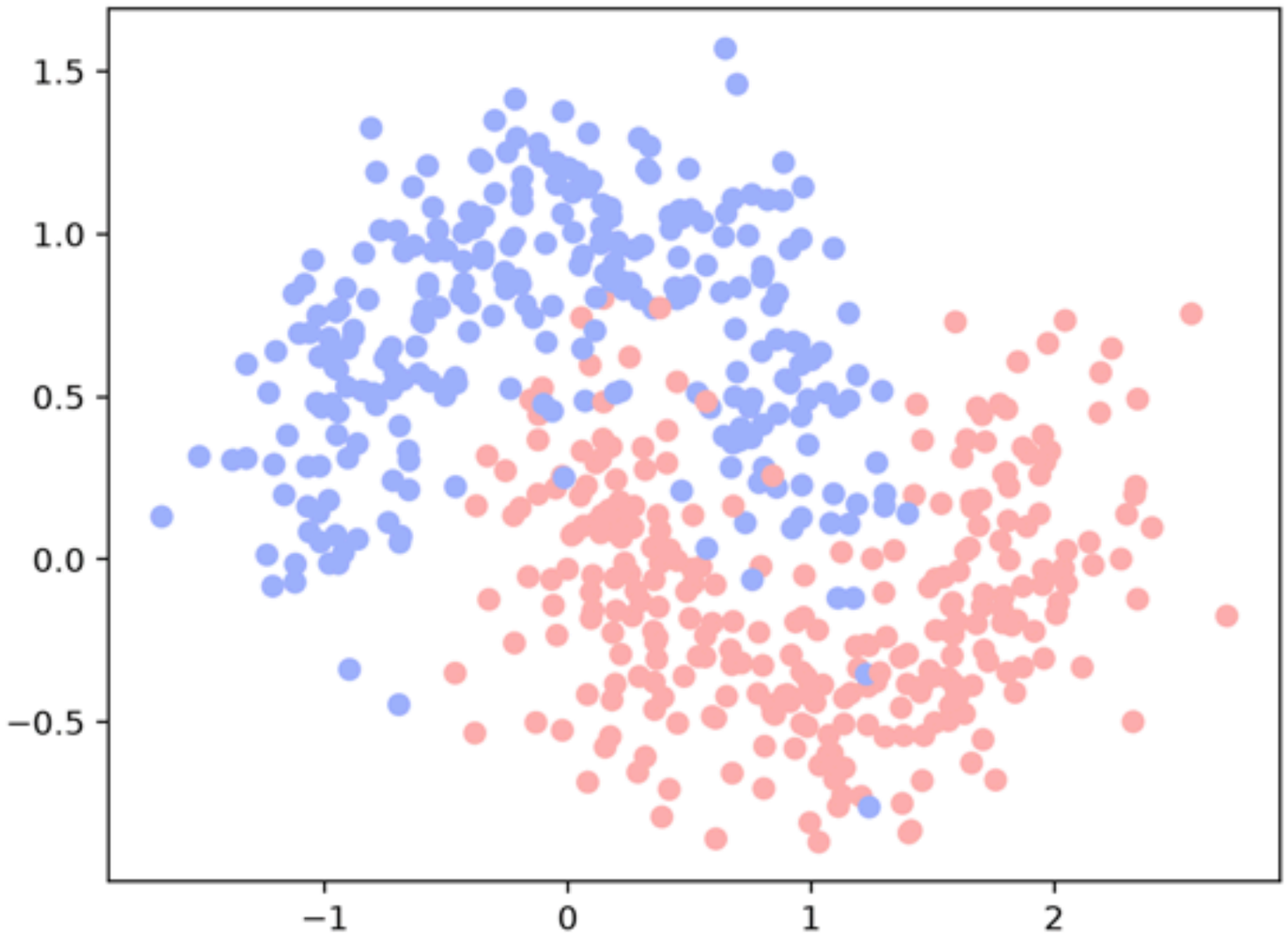
Heterogeneous Parallel Ensembles

- Create a synthetic data set

```
SECRET = 15121963

X, y = make_moons(1000, noise=0.25, random_state=SECRET)
X, Xval, y, yval = train_test_split(X, y, test_size=0.25)         #validatio
Xtrn, Xtst, ytrn, ytst = train_test_split(X, y, test_size=0.25)   #test
```

Heterogeneous Parallel Ensembles



Heterogeneous Parallel Ensembles

- Select a set of strong learners

```
models = {'dt': DecisionTreeClassifier(max_depth=5),
          'svm': SVC(gamma=1.0, C=1.0, probability=True),
          'gp': GaussianProcessClassifier(kernel=RBF(1.0)),
          #'5nn': KNeighborsClassifier(n_neighbors=5),
          '3nn': KNeighborsClassifier(n_neighbors=3),
          #'rf': RandomForestClassifier(max_depth=4,
          #       n_estimators=50),
          'rfsmall': RandomForestClassifier(max_depth=3,
                 n_estimators=50),
          'gnb': GaussianNB()
           }
```

Heterogeneous Parallel Ensembles

- We can see accuracy of the individual components on the test sets

```
def fit(models, X, y):
    for name in models:
        models[name].fit(X, y)
    return models

models = fit(models, Xtrn, ytrn)
```

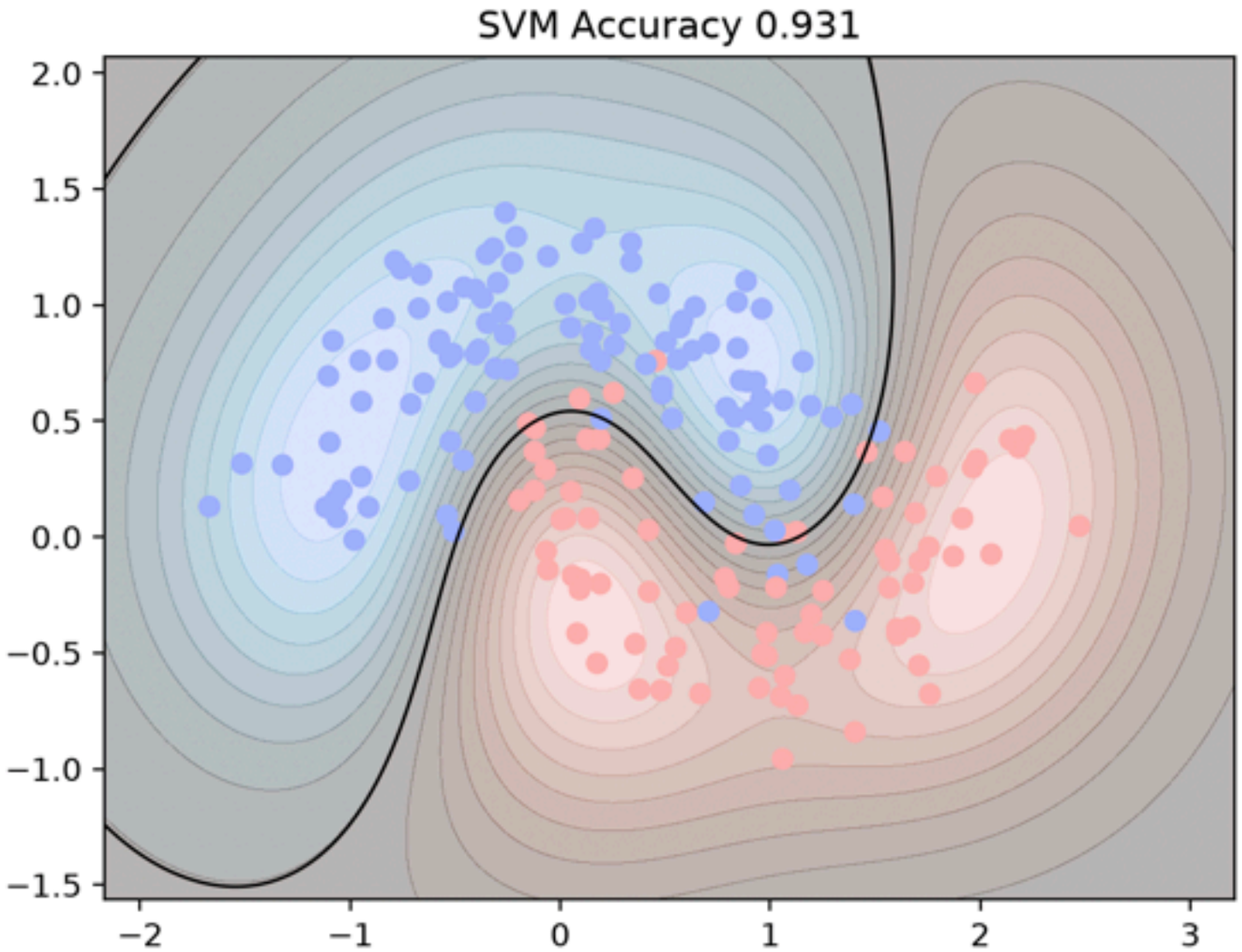
Heterogeneous Parallel Ensembles

```
def plot_model(clf, Xval, yval, title, res = 1000, pad=0.5 ):
    x0min, x0max = X[:, 0].min() - pad, X[:, 0].max() + pad
    x1min, x1max = X[:, 1].min() - pad, X[:, 1].max() + pad
    xx, yy = np.meshgrid(np.linspace(x0min, x0max, res),
                         np.linspace(x1min, x1max, res))
    Z = clf.predict(np.c [xx.ravel(), yy.ravel()]).reshape(xx.shape)

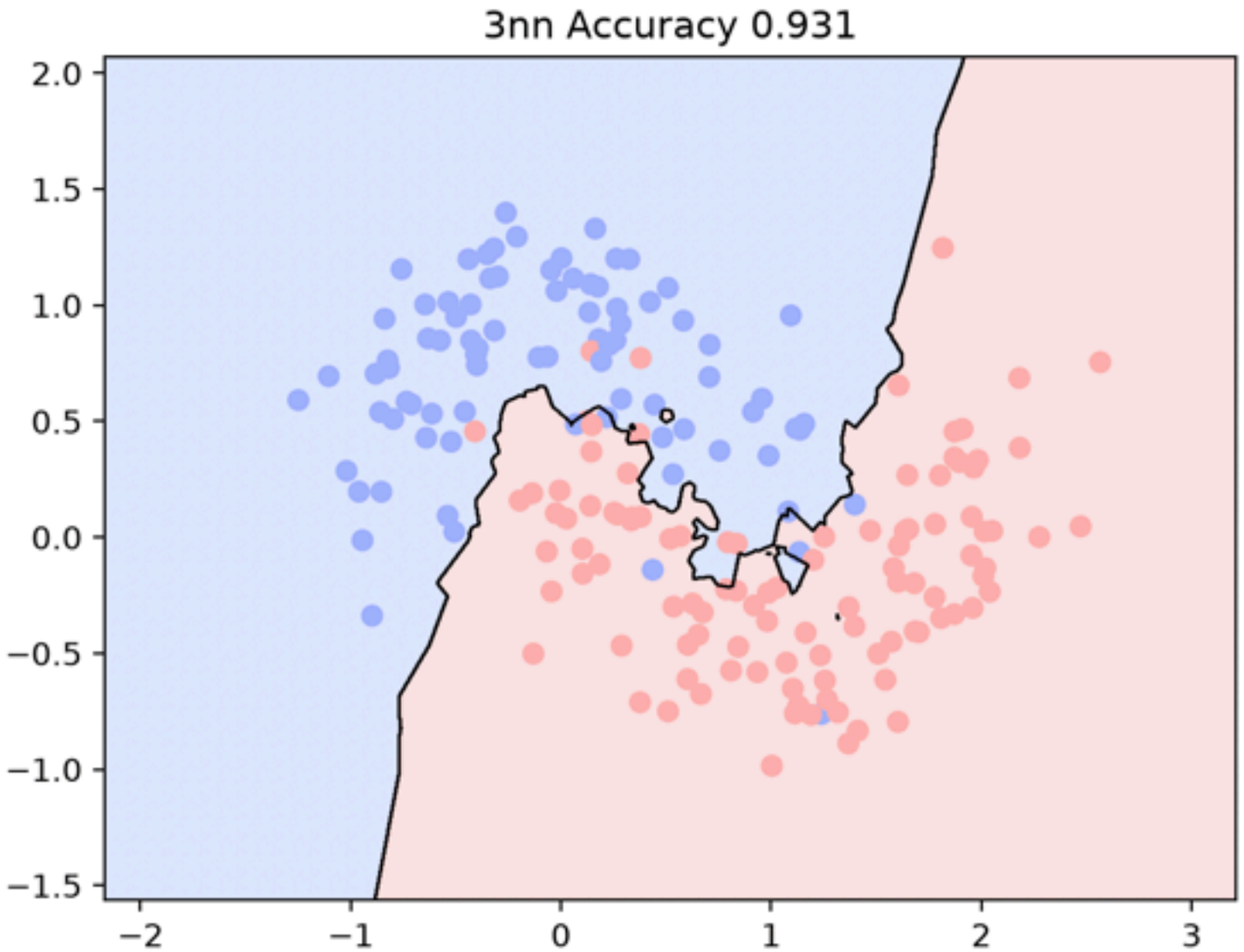
    plt.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 20),
                 cmap='berlin', alpha=0.3)
    plt.contour(xx, yy, Z, levels=[ 0], colors='k',
                linestyles=['-'], linewidths=[1])
    plt.scatter(Xval[:,0], Xval[:,1], c=yval, cmap='berlin')

    ypred = clf.predict(Xval)
    accuracy = round(accuracy_score(yval, ypred),3)
    plt.title(f'{title} Accuracy {accuracy}')
```

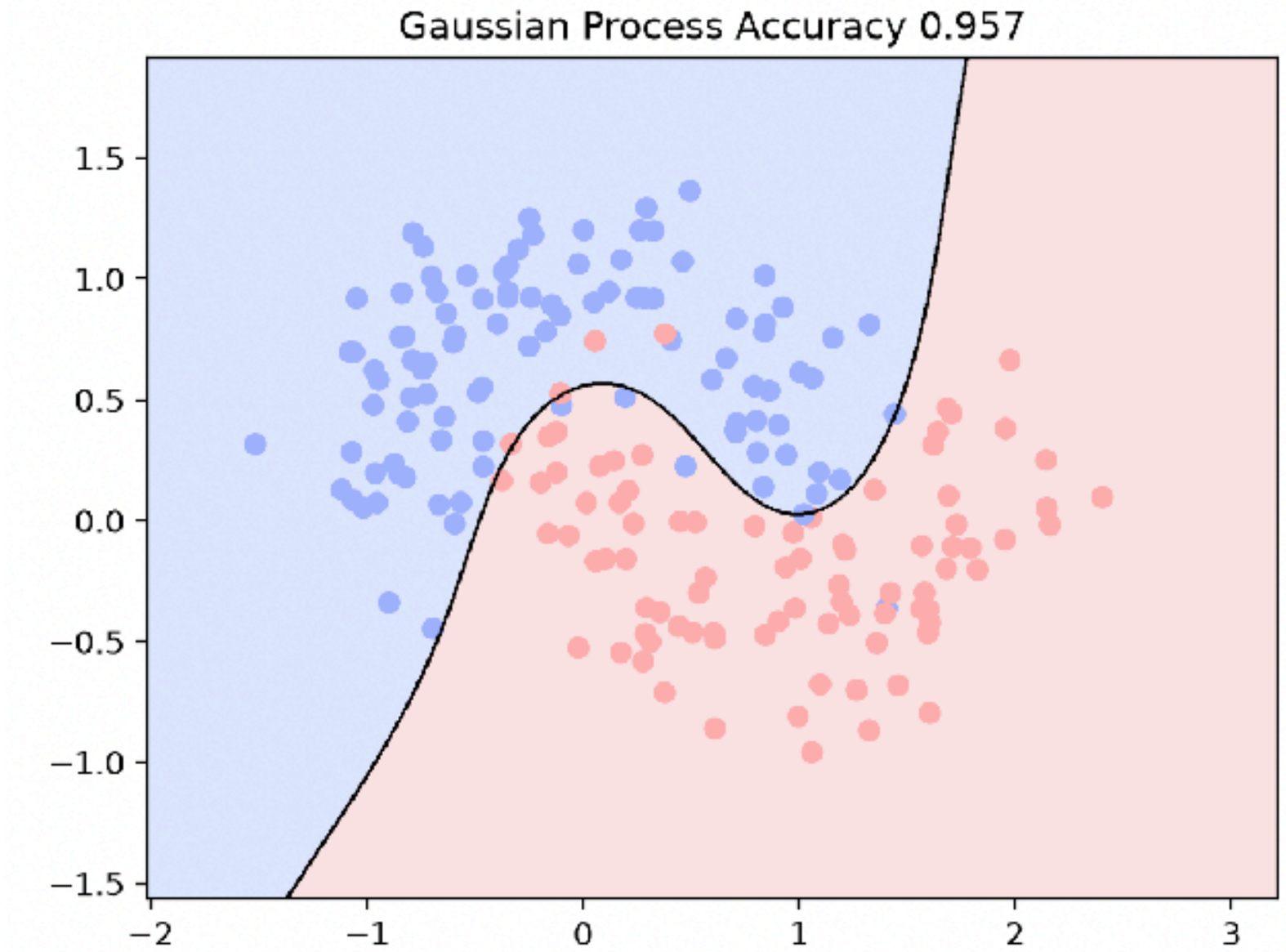
Heterogeneous Parallel Ensembles



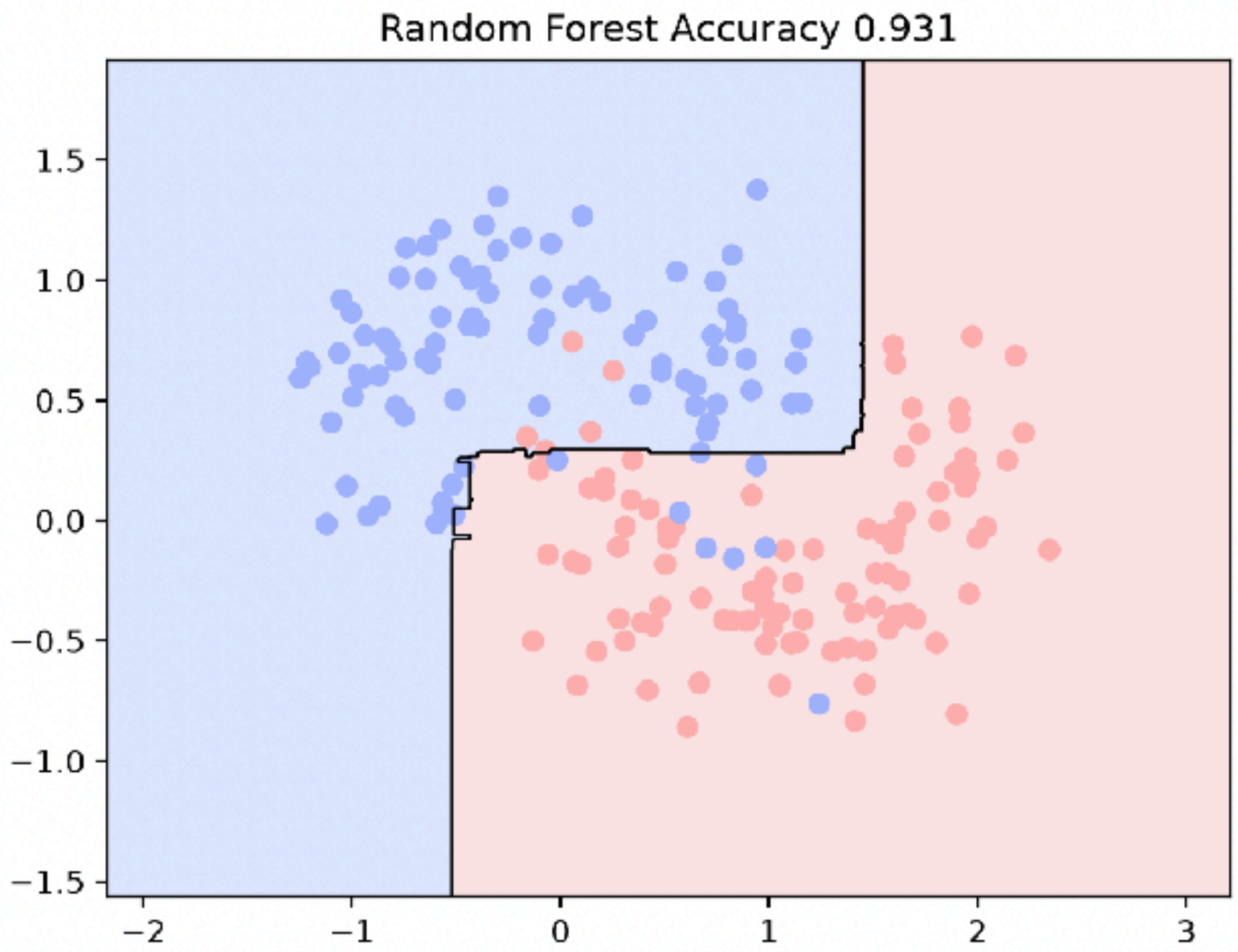
Heterogeneous Parallel Ensembles



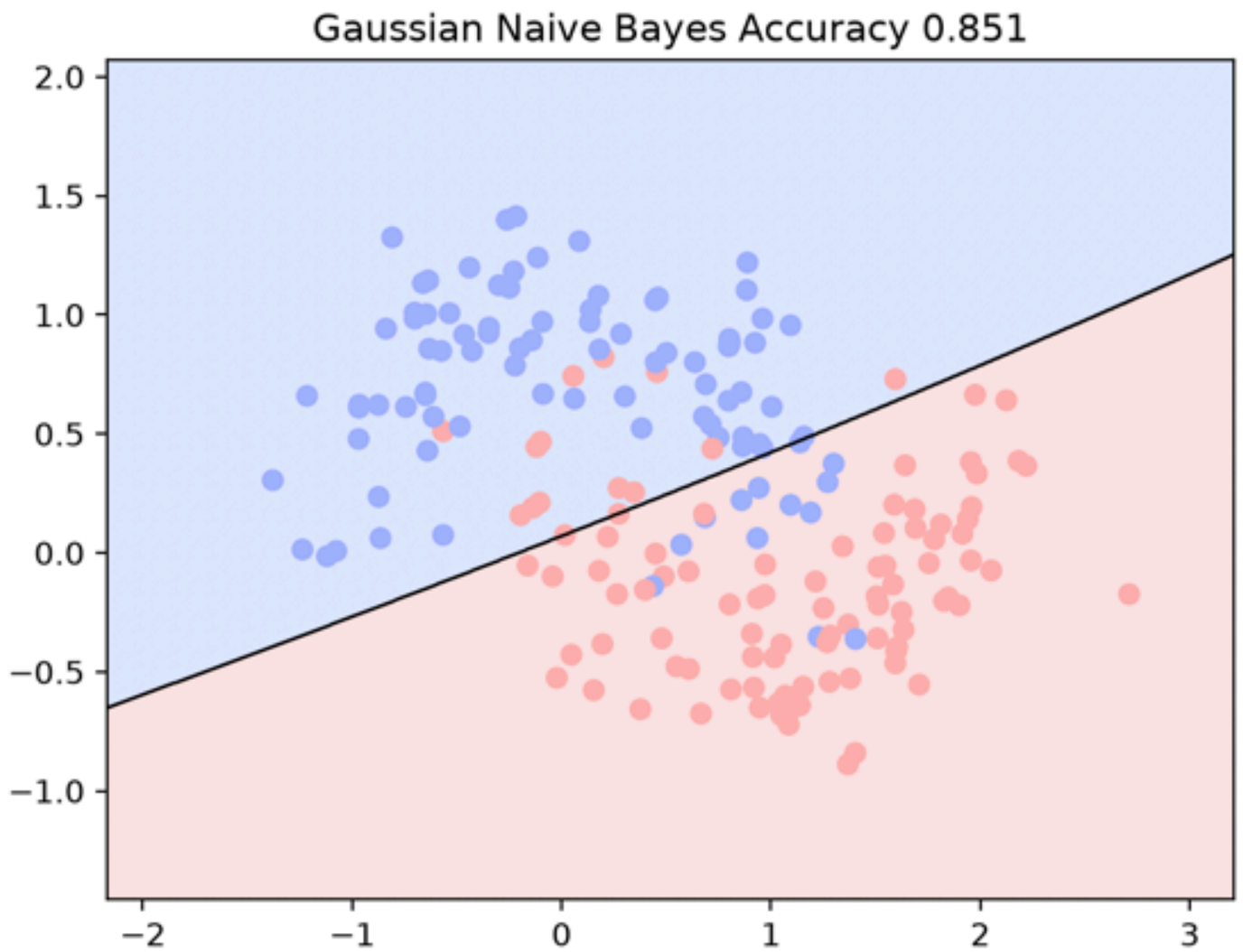
Heterogeneous Parallel Ensembles



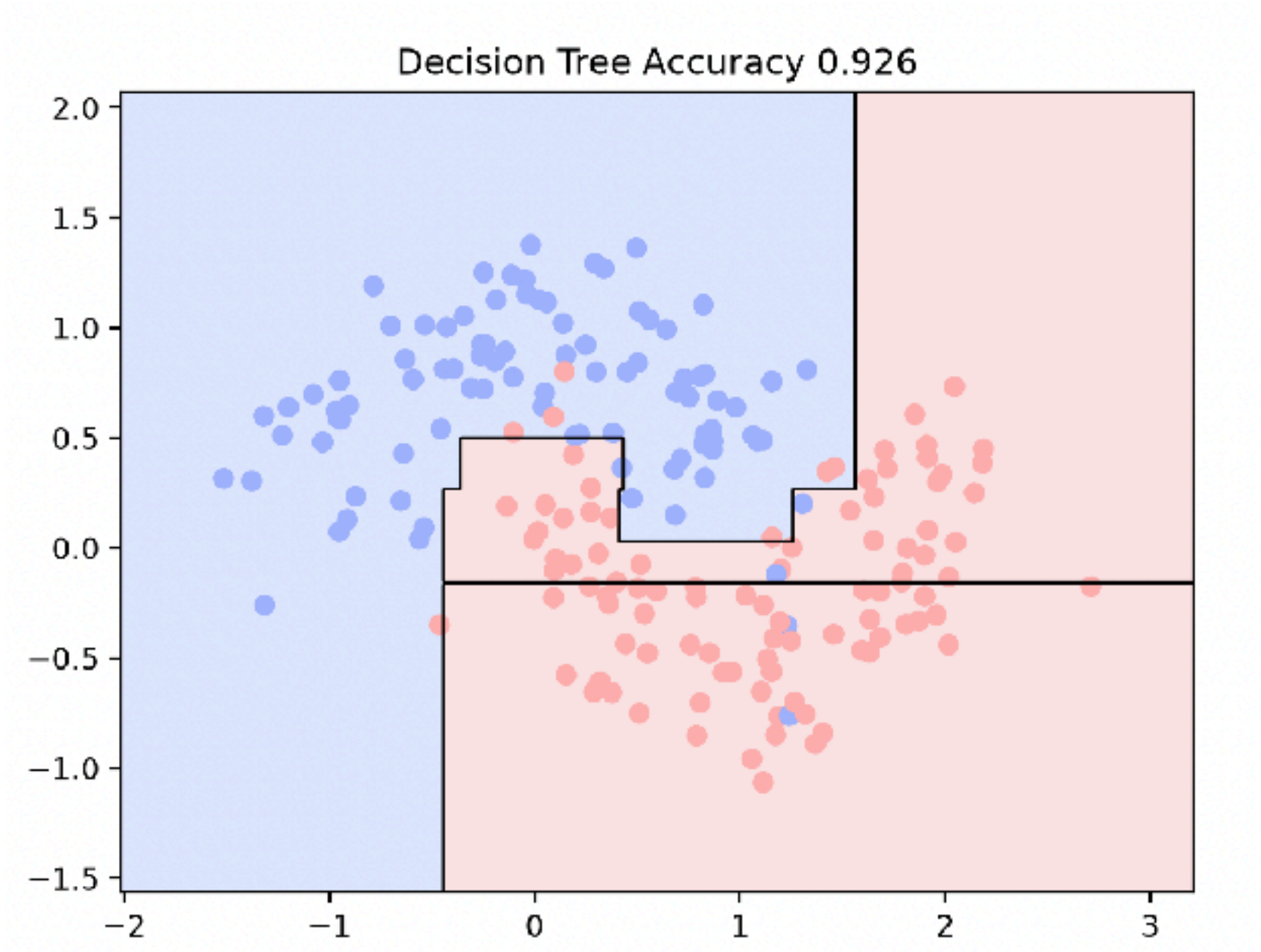
Heterogeneous Parallel Ensembles



Heterogeneous Parallel Ensembles



Heterogeneous Parallel Ensembles

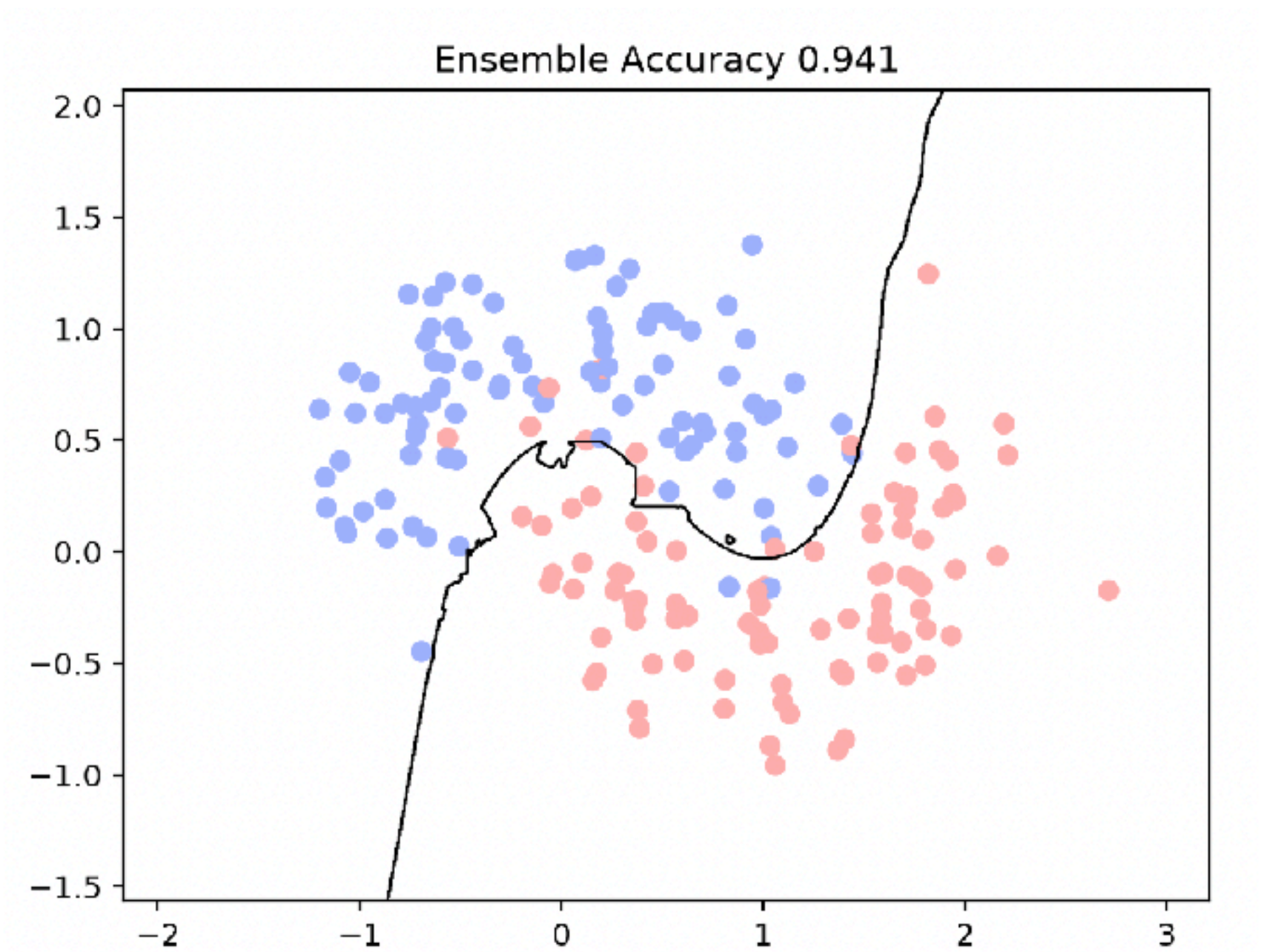


Heterogeneous Parallel Ensembles

- First method: Use majority voting
  - Can use mode from scipy.stats

```
def predict_union(models = models, X=Xtst):
    predictions = np.zeros((len(models), X.shape[0]))
    for index, model_name in enumerate(models):
        predictions[index,:] = models[model_name].predict(X)
    pred_final = mode(predictions, axis=0, keepdims=False)
    return pred_final[0].reshape(-1,)
```

Heterogeneous Parallel Ensembles



Heterogeneous Parallel Ensembles

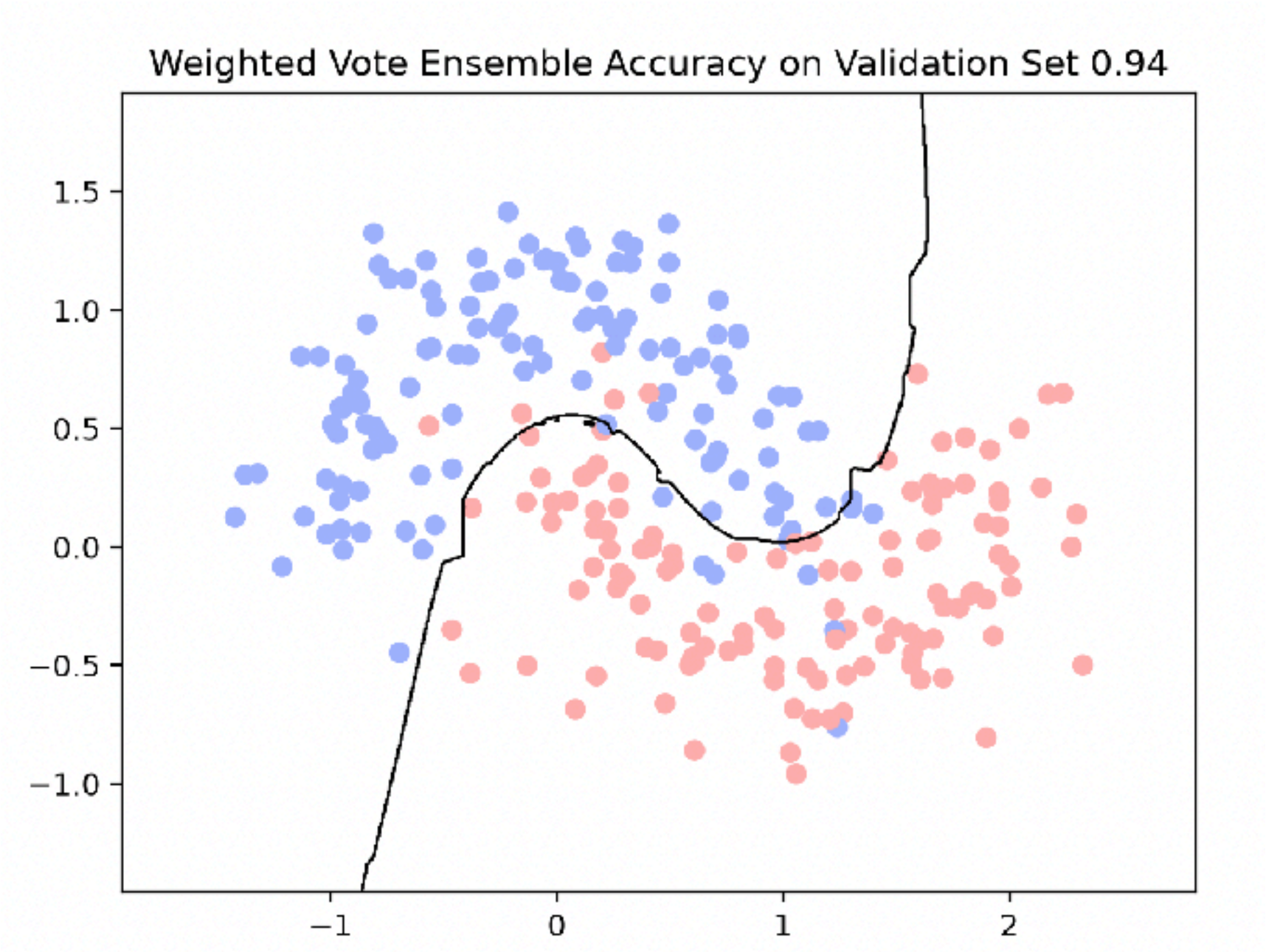
- Second is weighting
  - One idea is to use the accuracy on the test set to determine the weights w_i based on all the accuracies
  - $w_i = \frac{a_i}{\sum_{i=1}^{n} a_i}$
  - For this, we use the test sets for accuracies and then the validation set to see how it works

Heterogeneous Parallel Ensembles

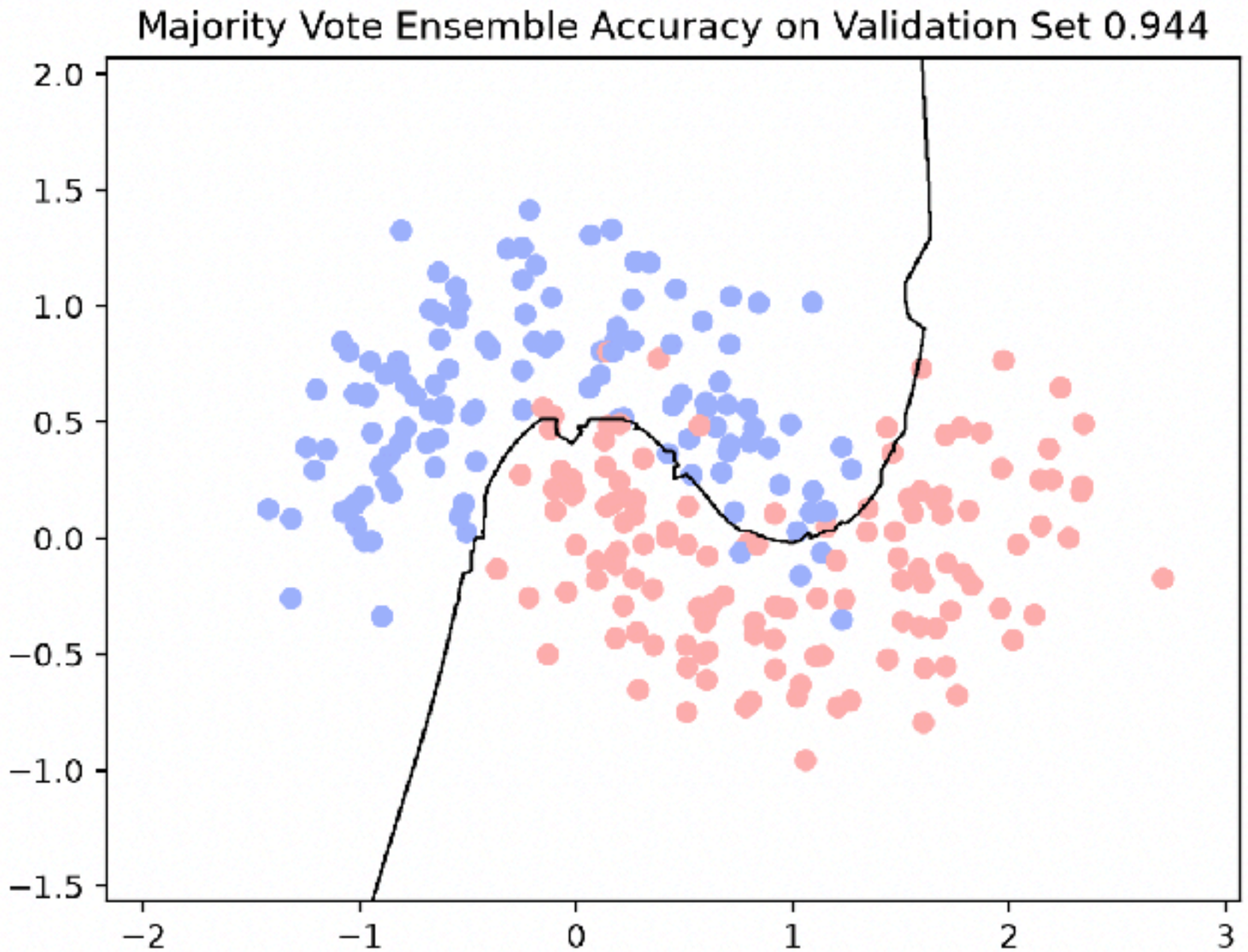
- Calculate weights, then use for prediction

```
def predict_union(models = models, Xtst=Xtst, Xval=Xval, ytst=ytst):
    predictions = np.zeros((len(models), Xtst.shape[0]))
    accuracies = np.zeros(len(models))
    for index, model_name in enumerate(models):
        predictions[index,:] = models[model_name].predict(Xtst)
        accuracies[index] = accuracy_score(predictions[index,:], ytst)
    weights = accuracies/sum(accuracies)
    predictions = np.zeros(( len(models), Xval.shape[0]))
    for index, model_name in enumerate(models):
        predictions[index,:] = models[model_name].predict(Xval)
    #print(predictions)
    pred_final = np.dot(weights,predictions)
    return np.where(pred_final>0.5, 1, 0)
```

Heterogeneous Parallel Ensembles

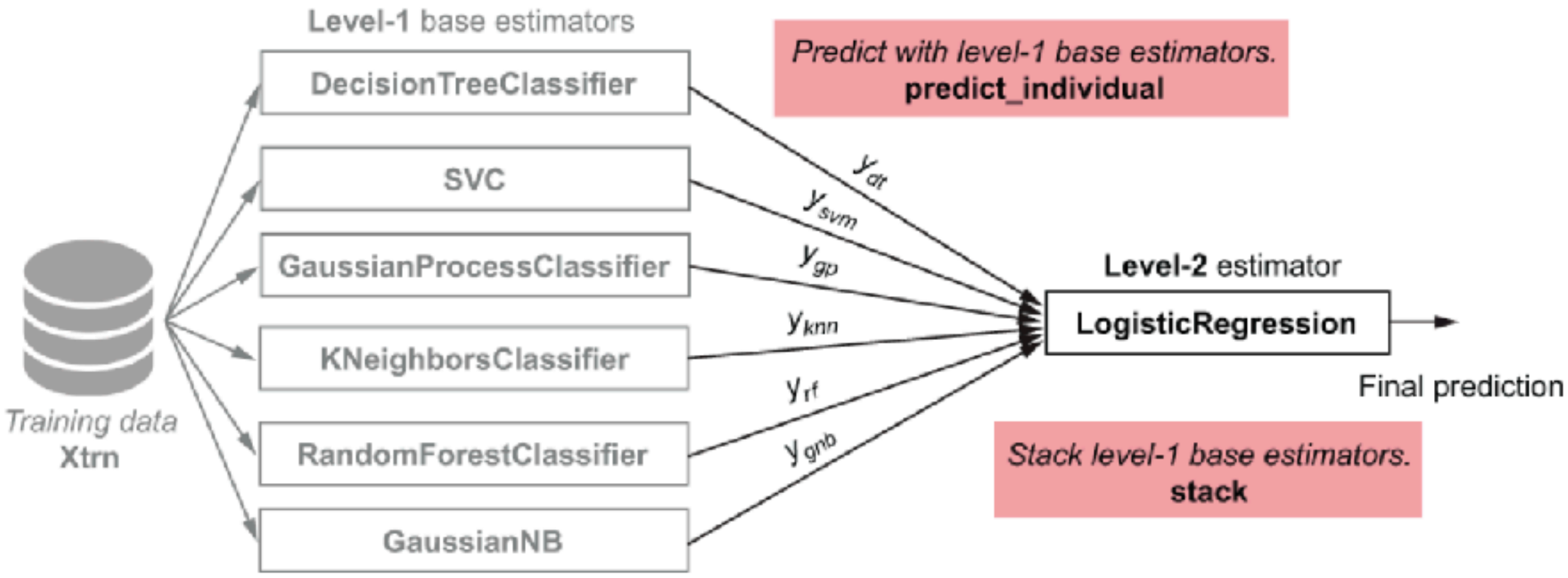


Heterogeneous Parallel Ensembles



Stacking

- Use a learner to learn the vote



Stacking

- Implemented in scikit-learn

```
from sklearn.ensemble import RandomForestClassifier, \
      GradientBoostingClassifier, StackingClassifier


meta_clf = LogisticRegression(max_iter=1000)

stacking_clf = StackingClassifier(
    estimators=[('dt', models['dt']), ('gp', models['gp']),
                ('svm', models['svm']), ('gnb', models['gnb']),
                ('rfsmall', models['rfsmall'])
                ],
    final_estimator=meta_clf,
    passthrough=True, # include original features with predictions
    cv=5
)
stacking_clf.fit(Xtrn, ytrn)
print(f"Stacking Accuracy: {stacking_clf.score(Xtst, ytst):.3f}")
print(f"Stacking Accuracy: {stacking_clf.score(Xval, yval):.3f}")
```

Stacking

- Uses usually 5-fold Cross Validation



Stacking

- Each fold consists of a training and a validation set
    - For each fold, we train using the training set and gather the results on the validation set
    - We then use the results on the validation sets of all folds to train the meta-learner



image-34

Adaptive Boosting

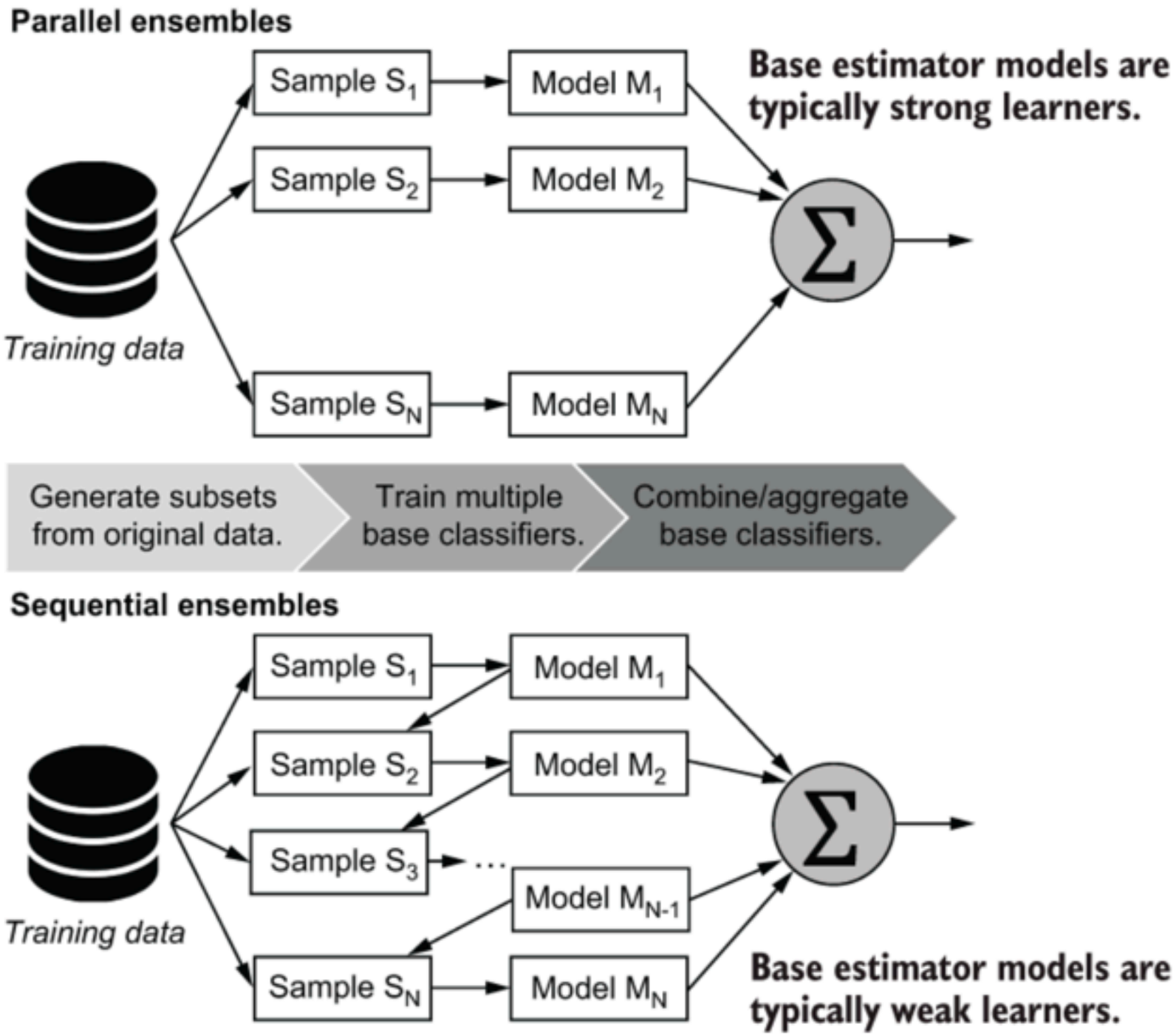
Adaptive Boosting

- Sequential ensembles:
  - exploit the dependence of base estimators
- Parallel ensembles:
  - exploit the independence of each base estimator

Adaptive Boosting

- Boosting combines weak learners
  - surprisingly simple to implement
  - computationally efficient
  - can be used with a wide variety of base-learning algorithms.

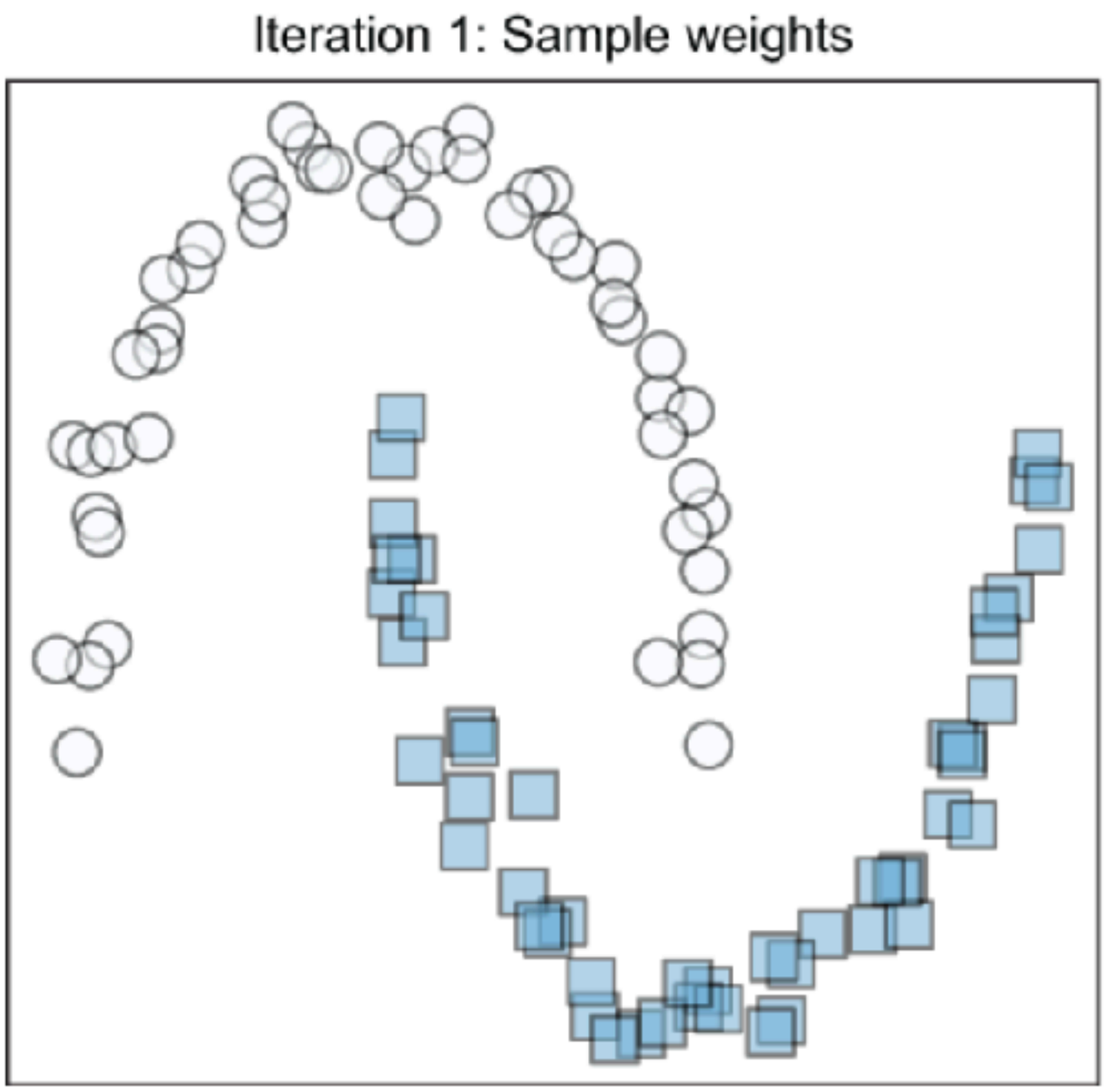
Adaptive Boosting



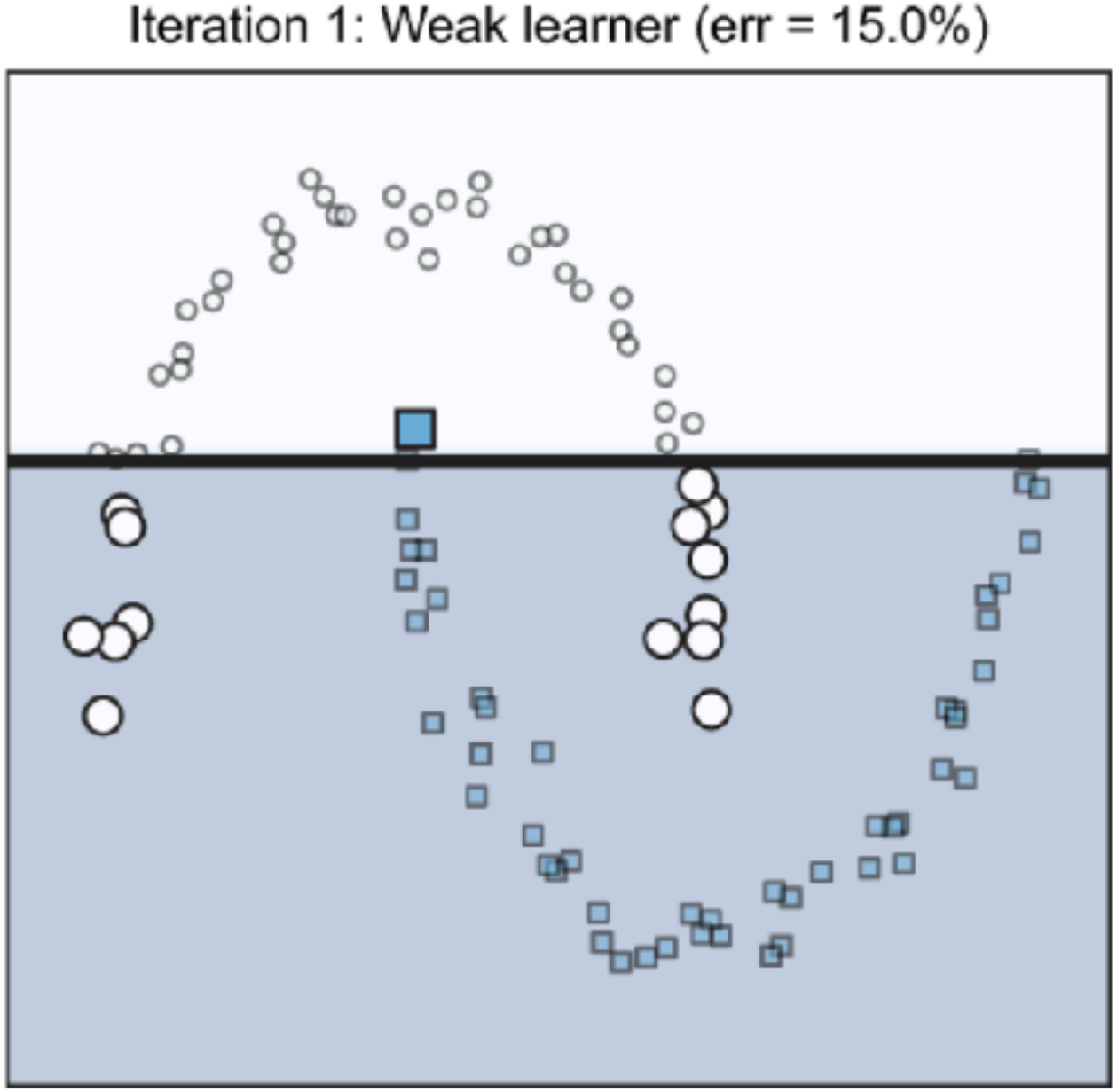
Adaptive Boosting

- Weak learner:
    - Performs better than random guessing.
- AdaBoost is an adaptive algorithm
    - At every iteration, it trains a new base estimator that fixes the mistakes made by the previous base estimator.

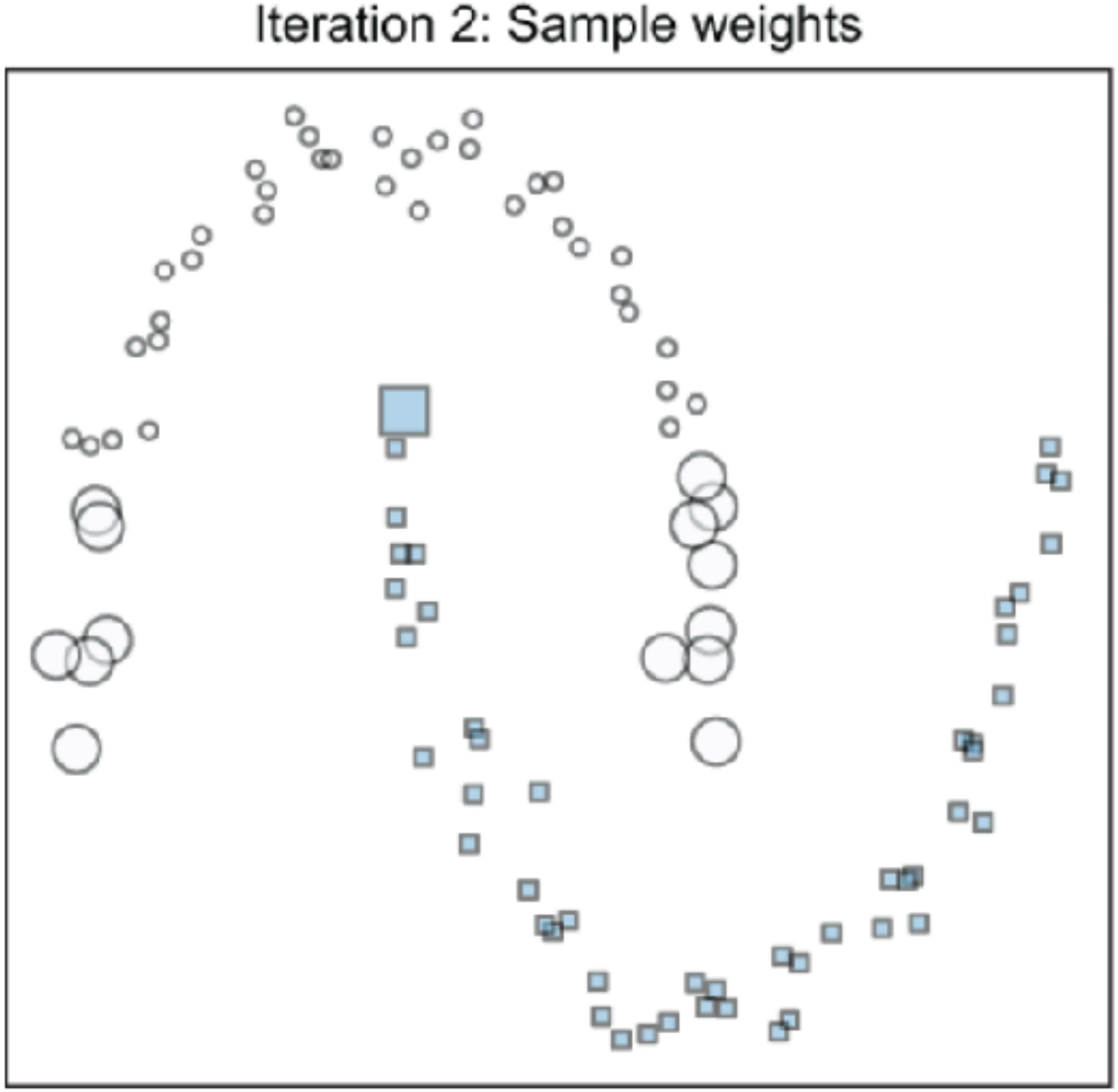
Adaptive Boosting



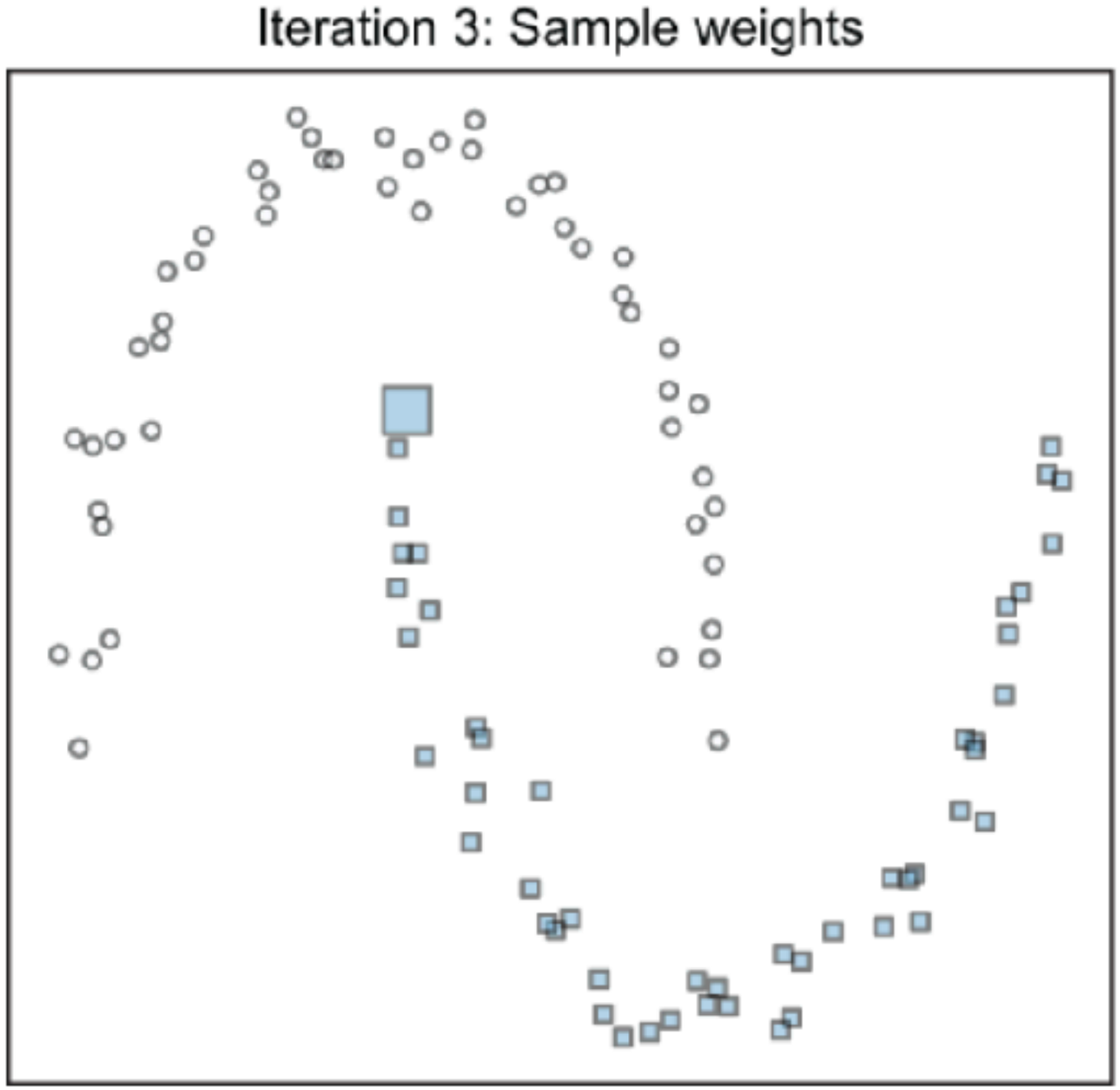
Adaptive Boosting



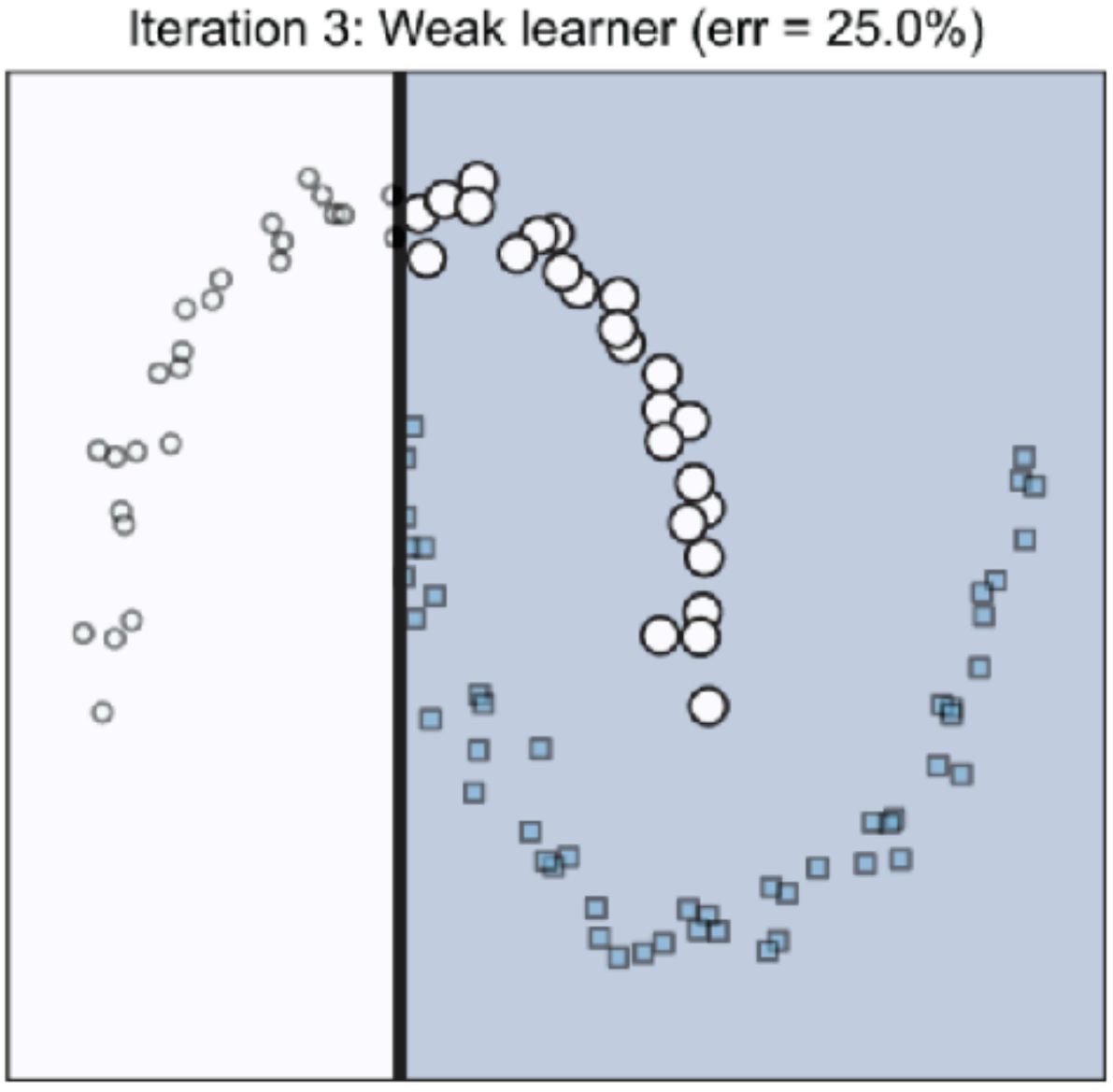
Adaptive Boosting



Adaptive Boosting

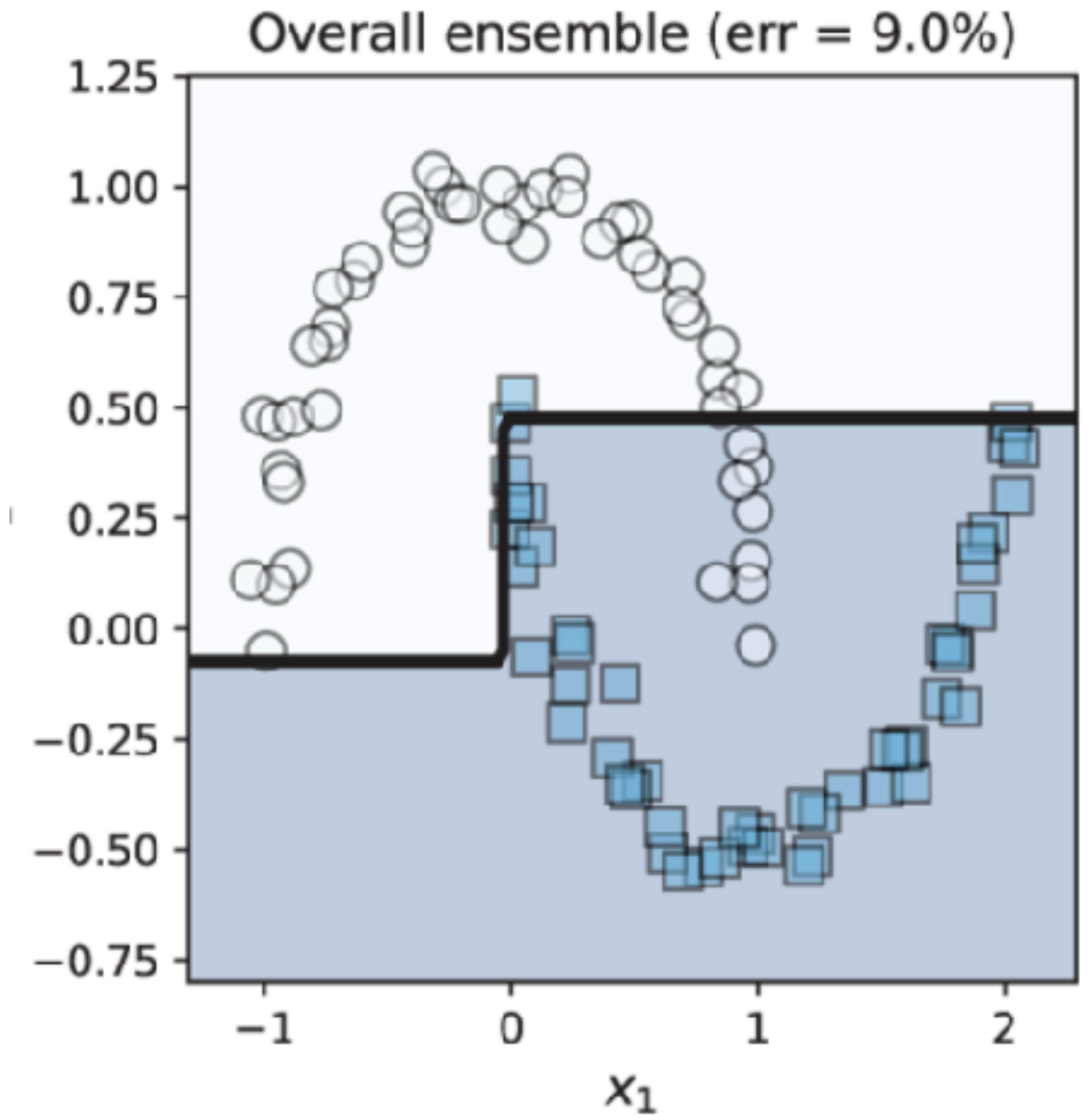


Adaptive Boosting



Adaptive Boosting

- Combine the weak learners into a meta-learner



Adaptive Boosting

- AdaBoost uses decision stumps as base estimators
    - can be trained extremely quickly, even with a large number of features.
    - Decision stumps are weak learners.
    - Contrast this to bagging, which uses deeper decision trees, that is, strong learners.

Adaptive Boosting

- AdaBoost keeps track of weights on individual training examples.
- Ensures ensemble diversity by reweighting training examples.
- Contrast this to bagging and random forests, which use resampling of training examples.

Adaptive Boosting

1. Train a weak learner $h_t(x)$ using the weighted training examples (x_i, y_i, D_i)

2. Compute the training error ε_t of the weak learner $h_t(x)$.

3. Compute the weight of the weak learner α_t that depends on ε_t.

4. Update the weights of the training examples, as follows:

    1. Increase the weight of misclassified examples by $D_i e^{\alpha_t}$.

    2. Decrease the weight of well classified examples by $D_i e^{\alpha_t}$.

AdaBoost

- AdaBoost
  - Set classifier to -1 and 1
  - Use *stumps* (very small trees)
    - Usually height = 1 (one decision) or 2 (three decisions)

AdaBoost

- Initial Weight Assignment:
  - Every data point has the same weight
- Train the first stump
- Calculate error ε as sum of the weights of all misclassified data points
- Model's influence

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \varepsilon}{\varepsilon} \right)$$

AdaBoost

- Weights are updated:
  - For misclassified points, weight w_{i+1} becomes
    - $w_{i+1} = e^{\alpha} \times w_i$
  - For correctly classified points,
    - $w_{i+1} = e^{-\alpha} \times w_i$
- Normalize all weights (by dividing by the sum of weights)

AdaBoost

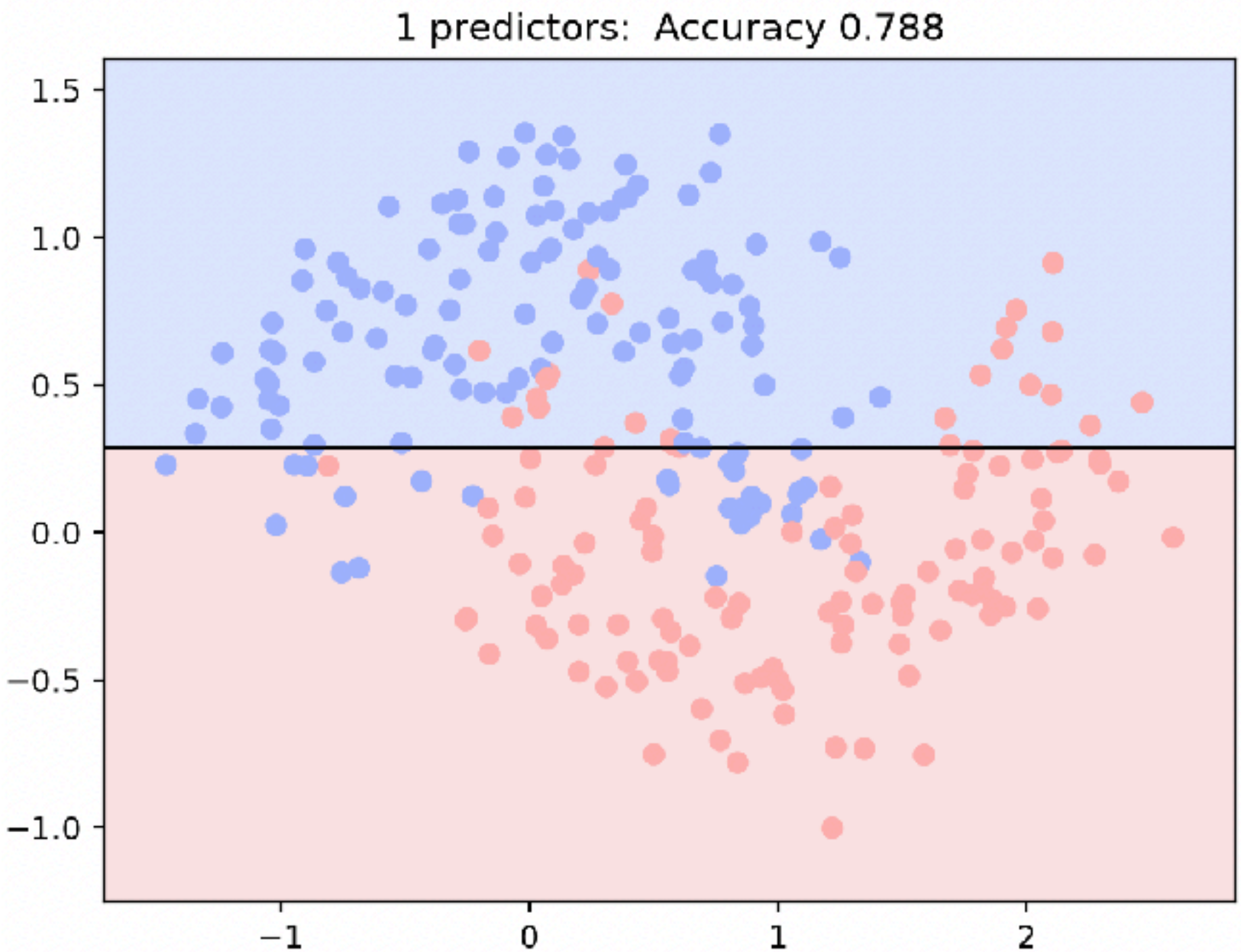
- Up-sample:
  - Use the new weights to sample for the next weak lerner
- Repeat many times
  - …
- Use a learning factor in order to shrink already established weights

AdaBoost

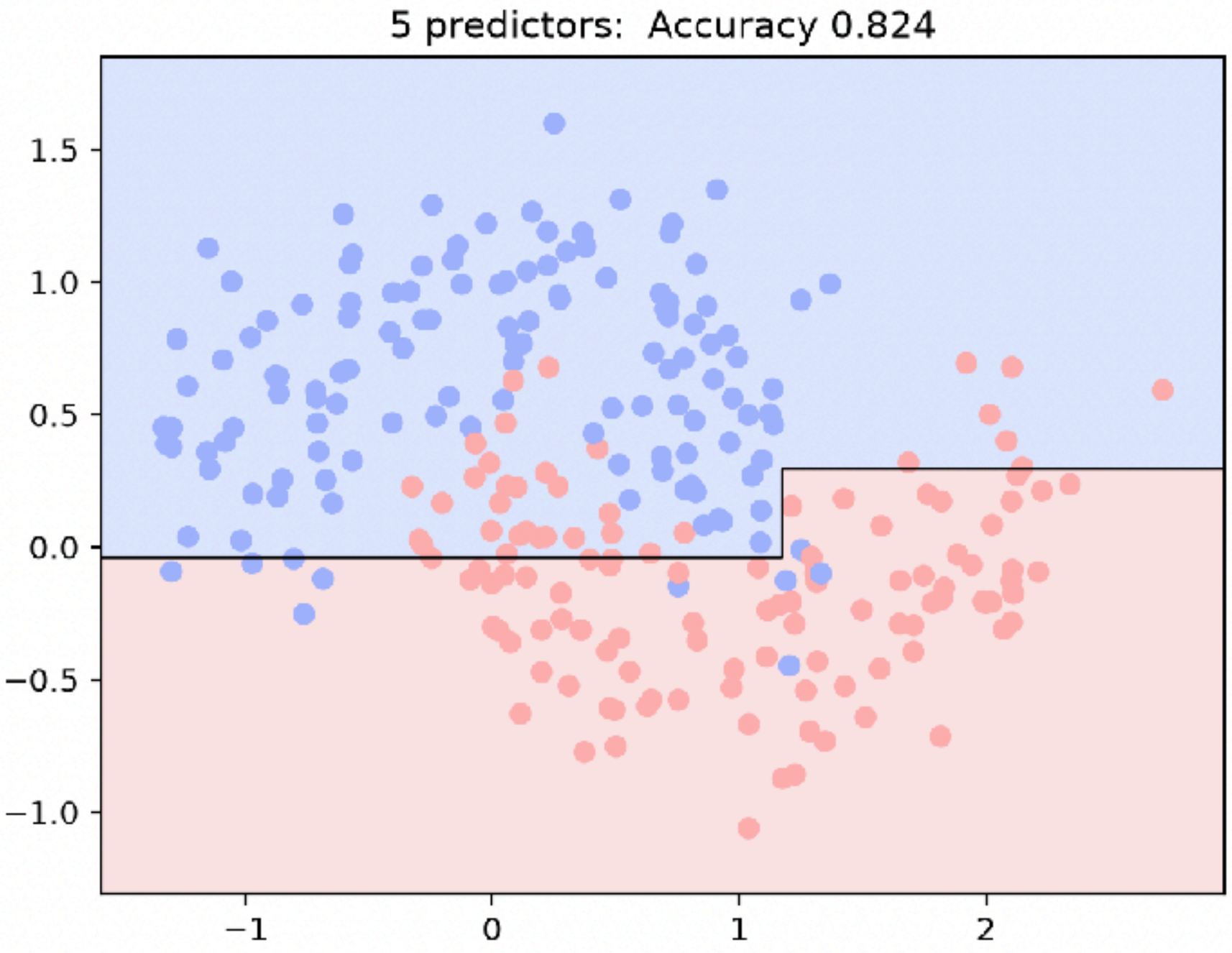
- Final prediction

$$\text{sign}\left(\sum_{i=1}^{n} \alpha_i \times \text{decision}_i(x)\right)$$

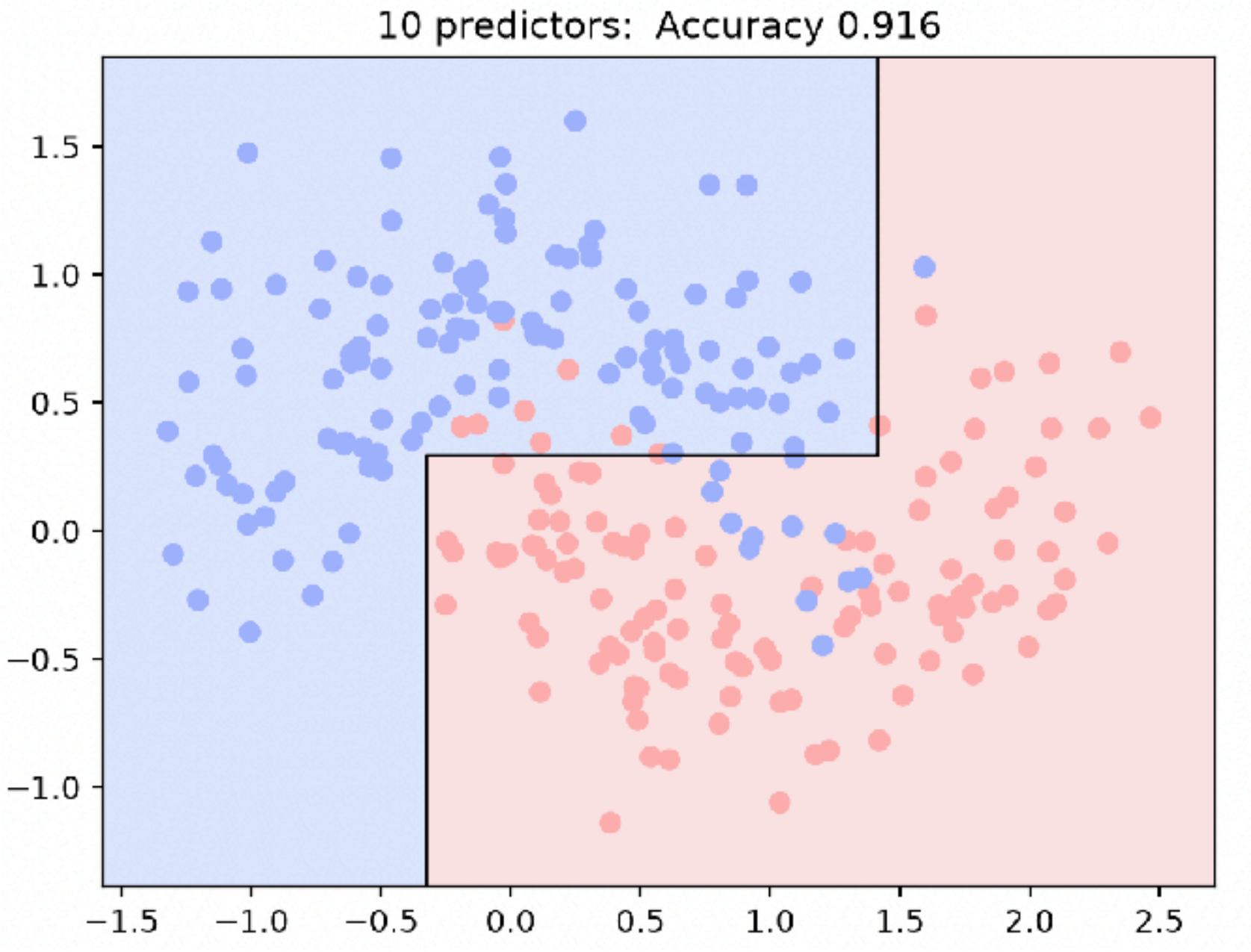
AdaBoost Example



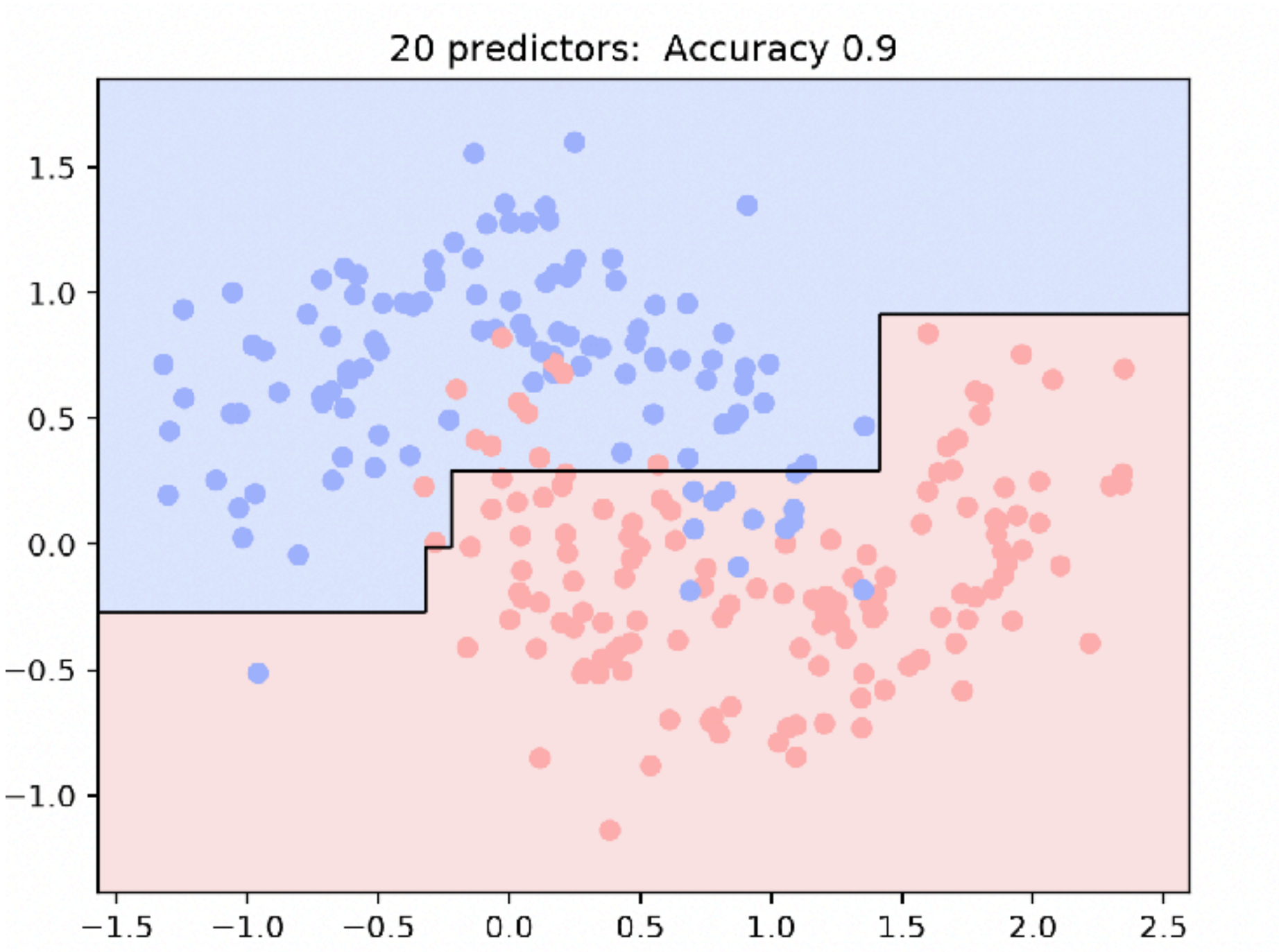
AdaBoost Example



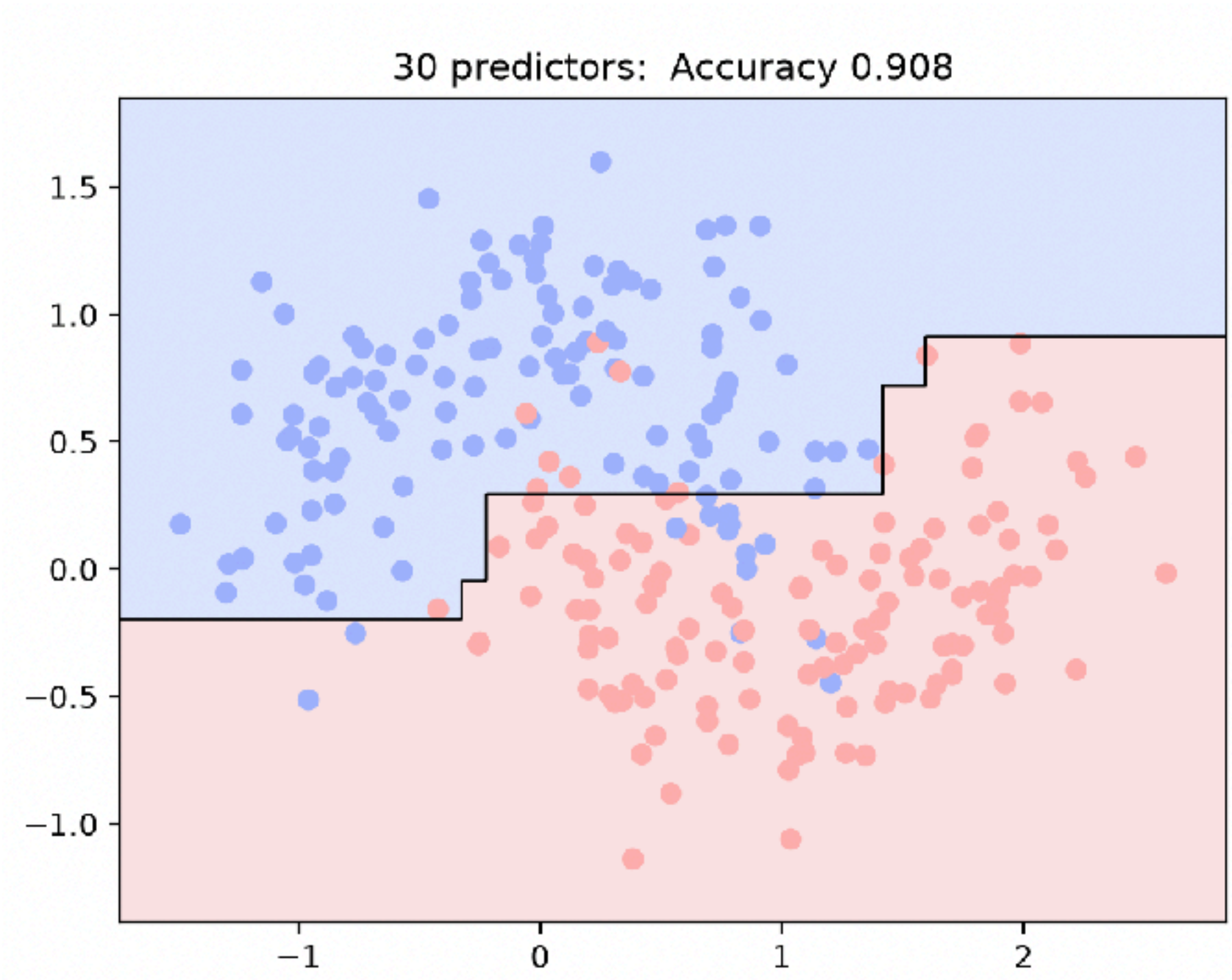
AdaBoost Example



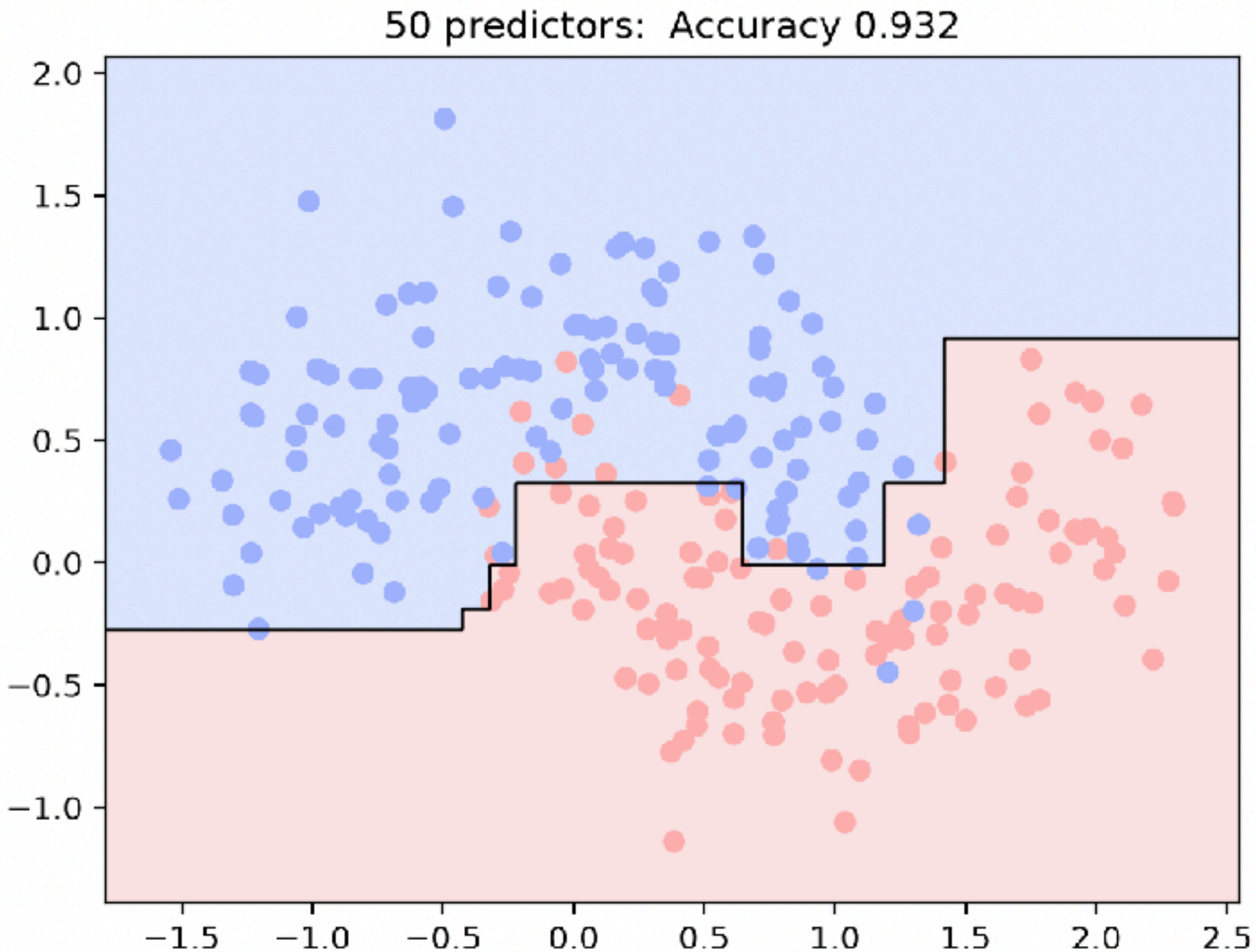
AdaBoost Example



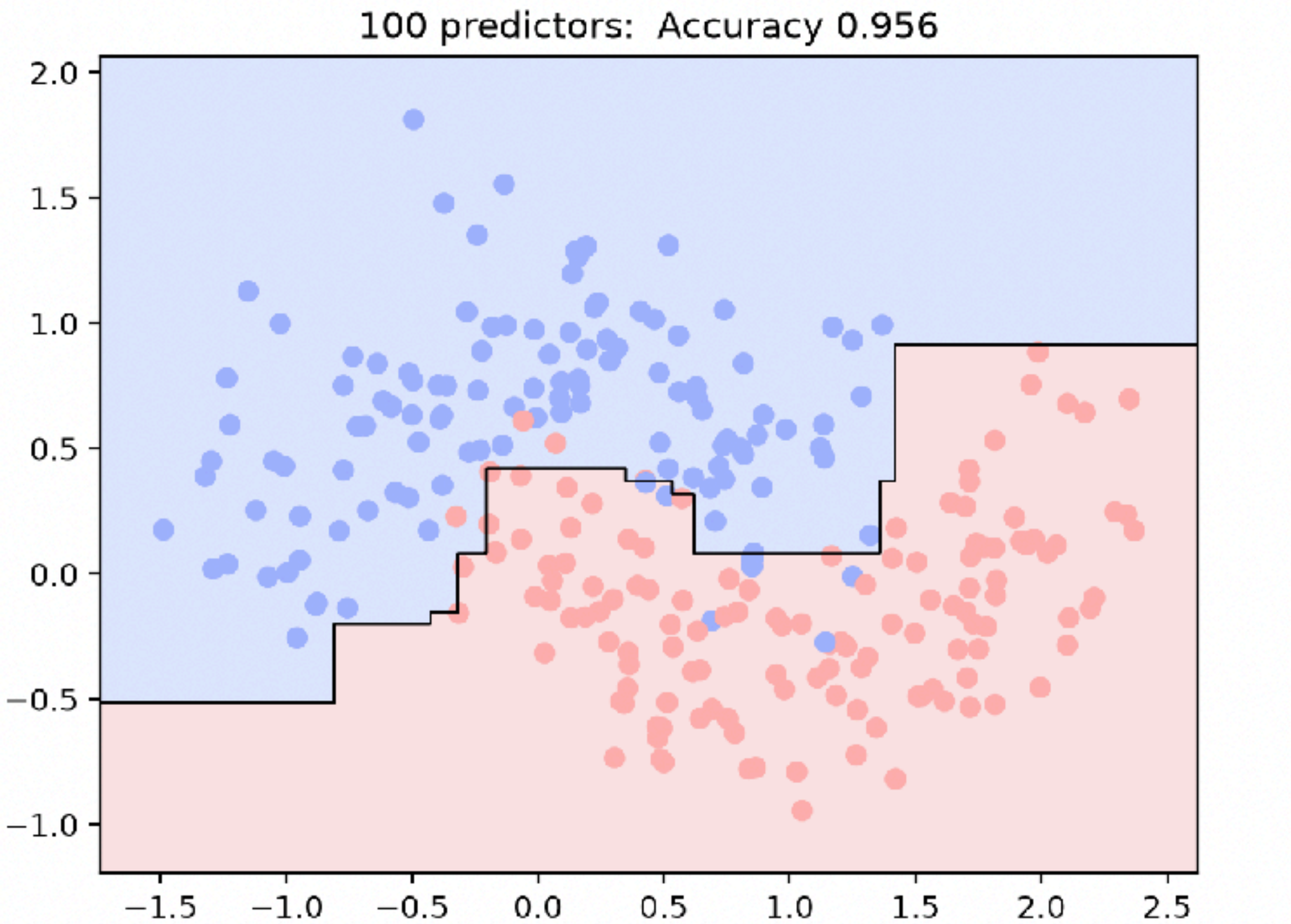
AdaBoost Example



AdaBoost Example



AdaBoost Example





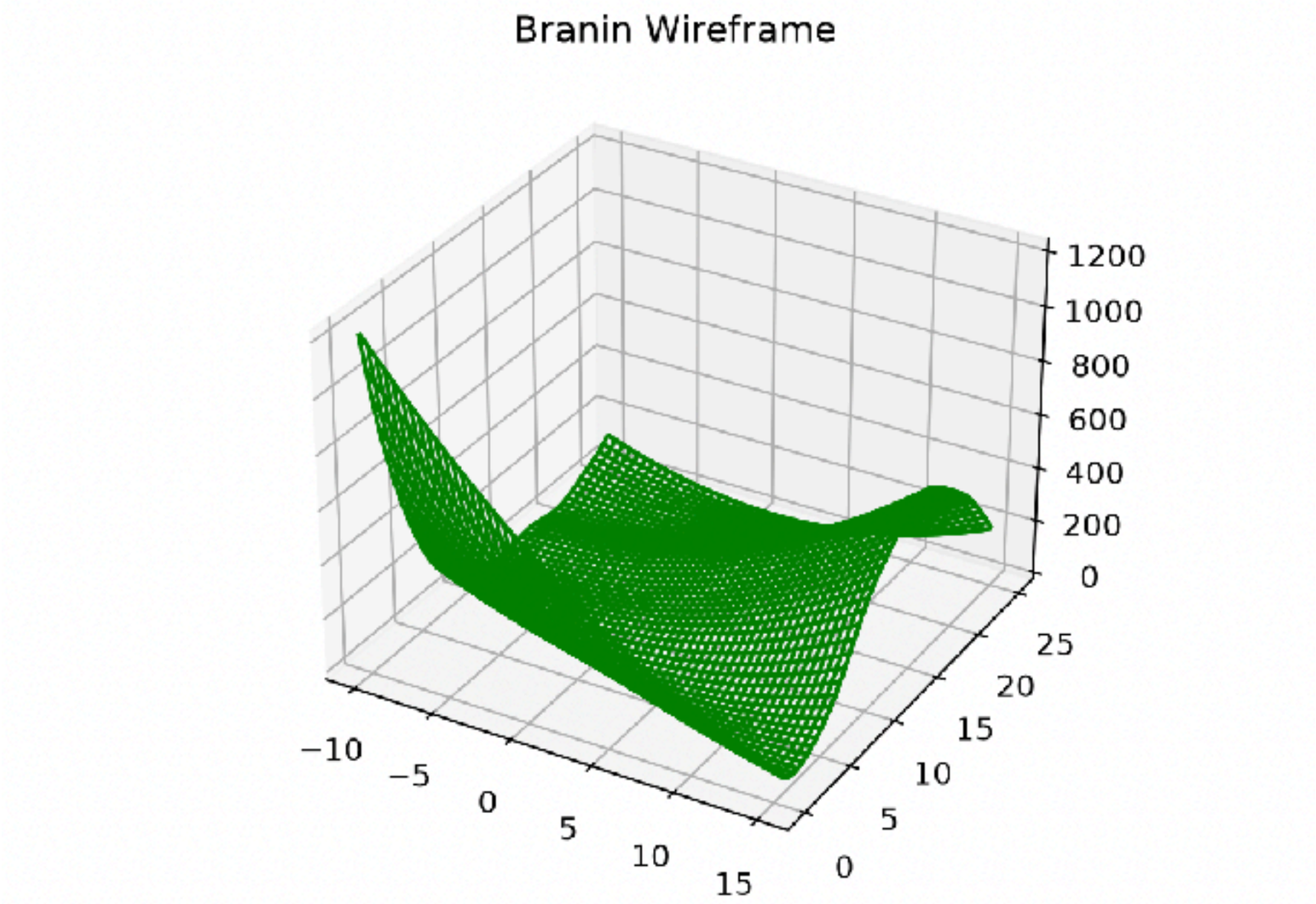
Gradient Boosting

Gradient Boosting

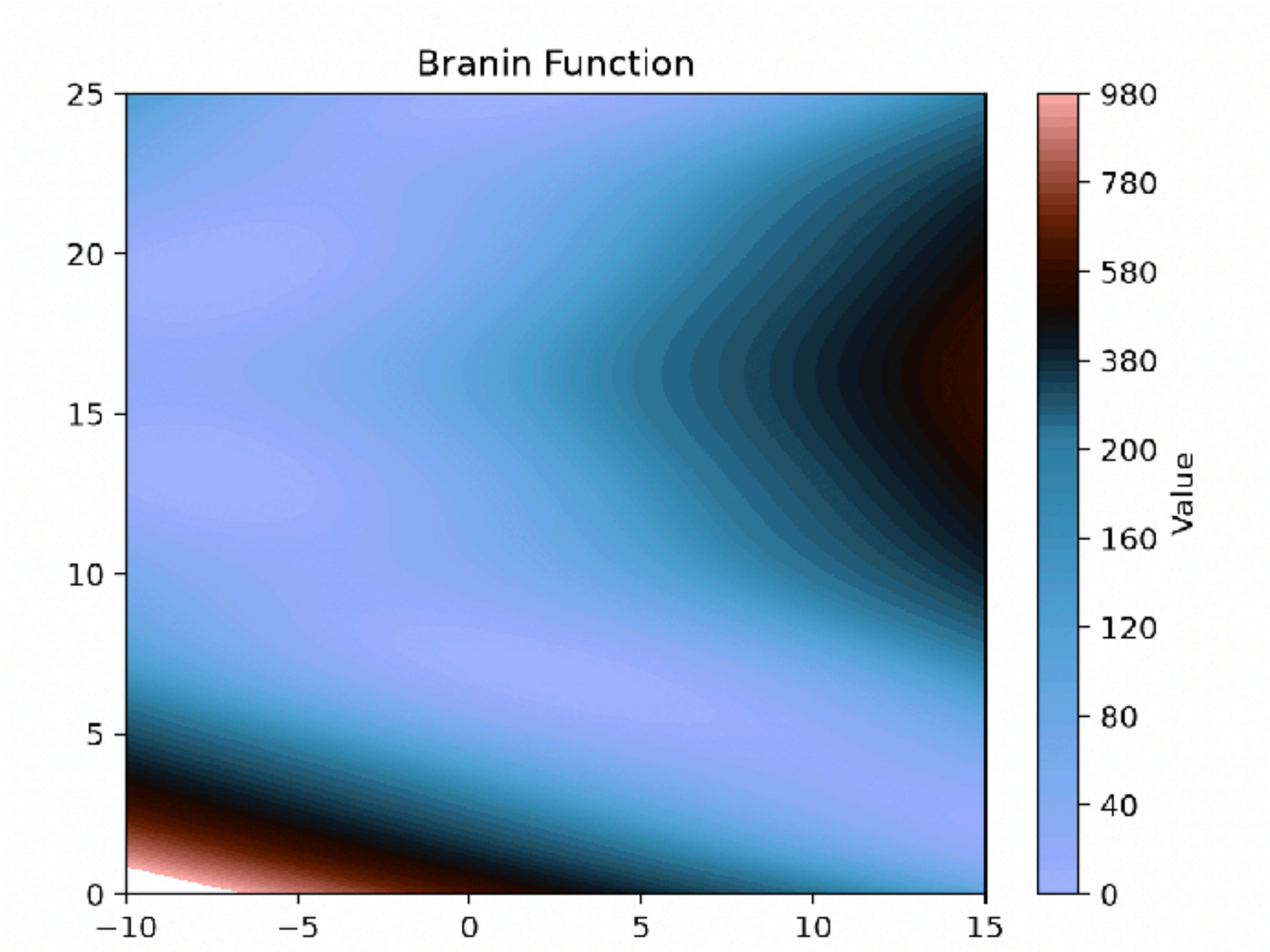
- We use the Branin function:
  -

```
def branin(x):
    x1, x2 = x[0], x[1]
    a, b, c = 1.0, 5.1 / (4 * np.pi**2), 5.0 / np.pi
    r, s, t = 6.0, 10.0, 1.0 / (8 * np.pi)
    return a*(x2-b*x1**2+c*x1-r)**2+s*(1-t)*np.cos(x1)+s
```

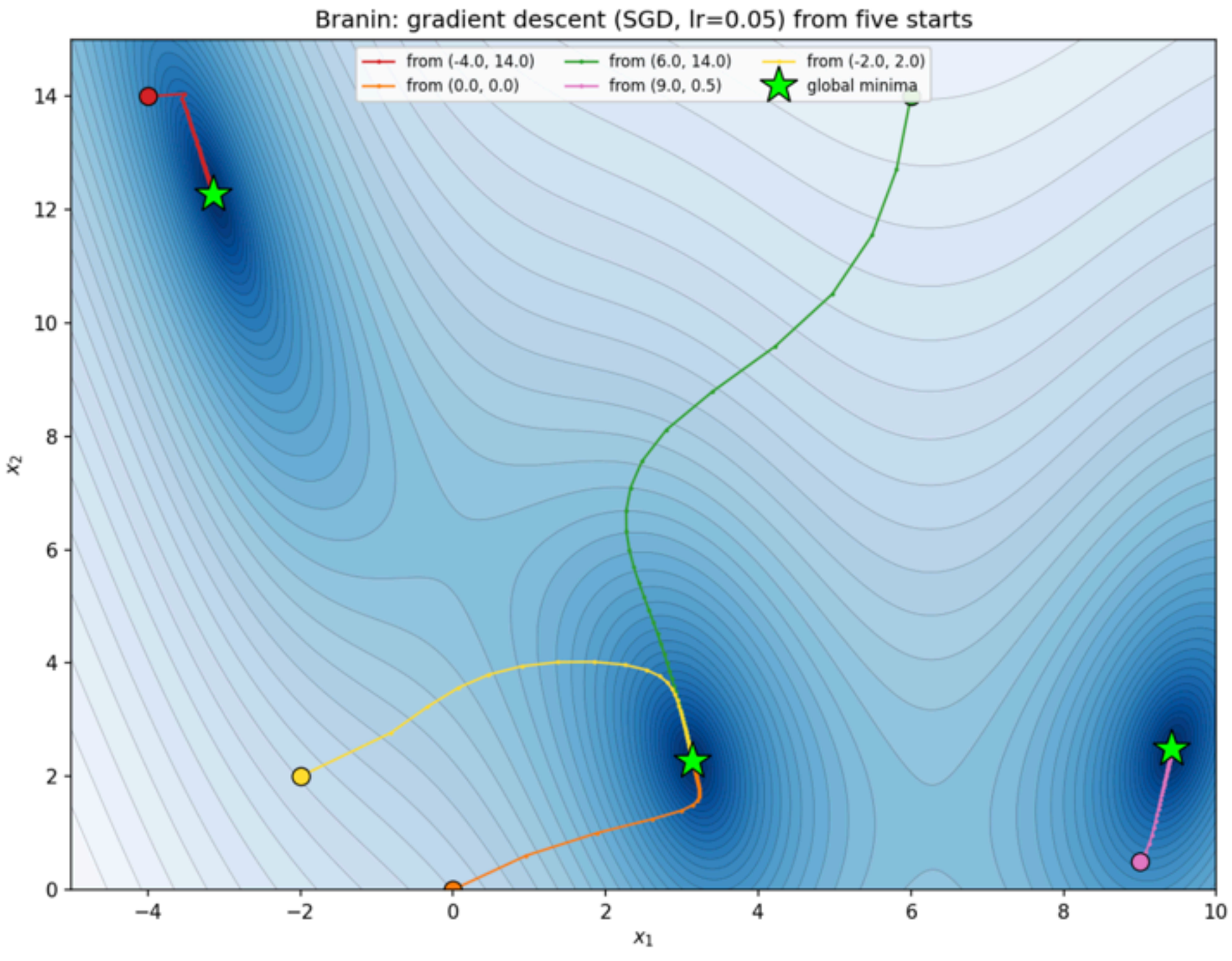
Gradient Boosting



Gradient Boosting



Gradient Boosting



Gradient Descent

- Minimize $f : \mathbb{R}^d \longrightarrow \mathbb{R}$
  - Start at a given point x
  - Calculate ∇f at x
  - Determine the point on the line $\{x + t\nabla f \mid t \in \mathbb{R}\}$ that minimizes f. Continue from there

Gradient Descent

- We then determine the point on the line that minimizes f
  - There are a variety of methods
    - We can use the derivative
      - This is dangerous because the derivative can be very small
  - We can use Brent's method
    - Finds three points with $a < b < c; f(a) > f(b) < f(c)$
    - Divide the interval between a and b as well as b and c and find tighter points with this property

Gradient Descent

```
def nabla_branin(w):
    a, b, c = 1.0, 5.1 / (4 * np.pi**2), 5.0 / np.pi
    r, s, t = 6.0, 10.0, 1.0 / (8 * np.pi)
    return np.array([2*a*(w[1]-b*w[0]**2+c*w[0]-r)
                        *(-2*b*w[0]+c)-
                     s*(1-t)*np.sin(w[0]),
                     2*a*(w[1]-b*w[0]**2+c*w[0]-r)])
```

Gradient Descent

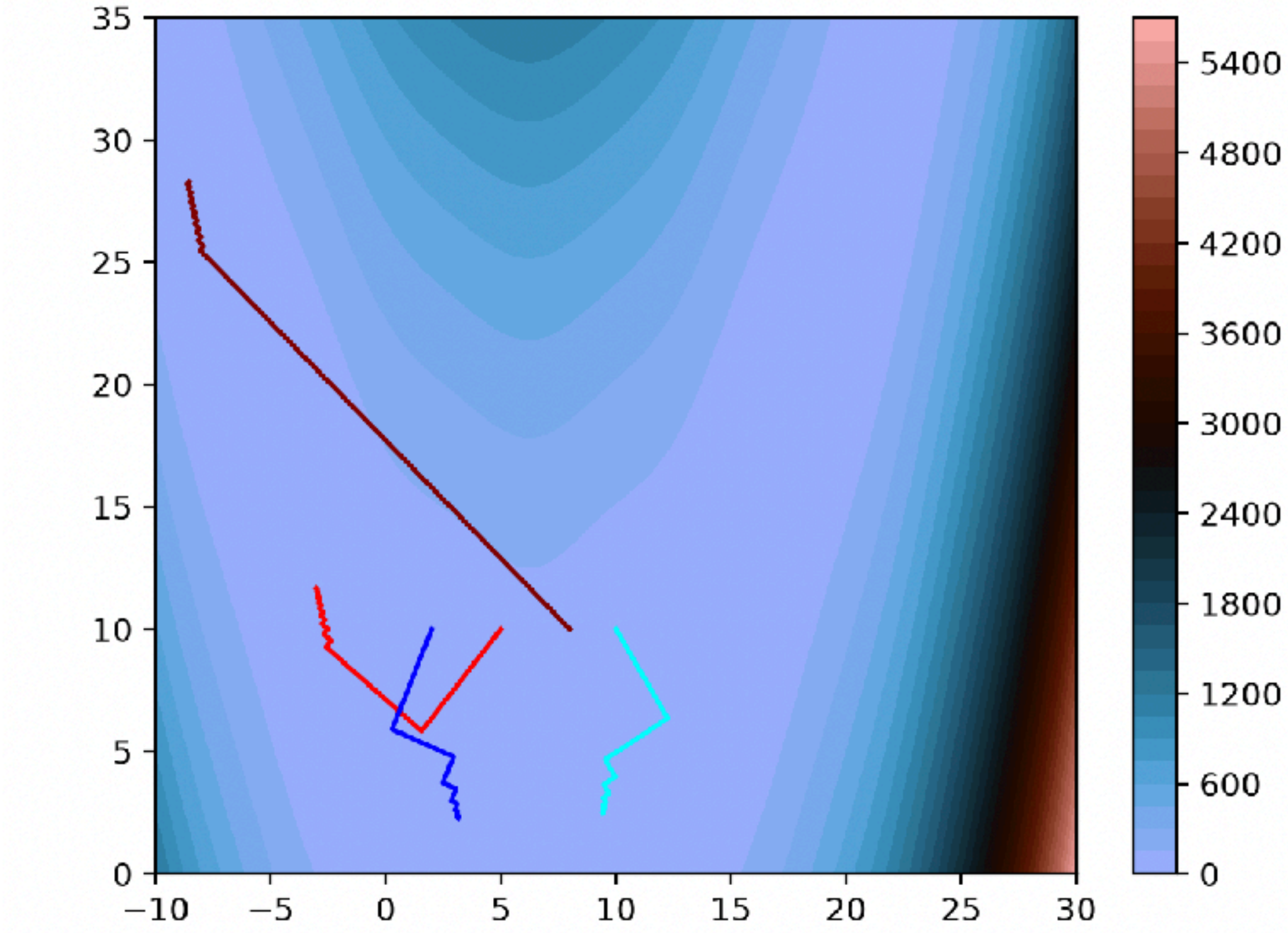
```
pts = [np.array([5,15])]
for _ in range(20):
    grad = nabla_branin(pts[-1])
    on_line = lambda t: branin(pts[-1]-grad*t)
    step = minimize_scalar(on_line, method='golden').x
    pts.append(pts[-1]-step*grad)
plt.plot([pt[0] for pt in pts],
         [pt[1] for pt in pts],
          c = 'red')
```

Gradient Descent

```
def nabla_branin(w):
    a, b, c = 1.0, 5.1 / (4 * np.pi**2), 5.0 / np.pi
    r, s, t = 6.0, 10.0, 1.0 / (8 * np.pi)
    return np.array([2*a*(w[1]-b*w[0]**2+c*w[0]-r)
                  *(-2*b*w[0]+c)-s*(1-t)*np.sin(w[0]),
                      2*a*(w[1]-b*w[0]**2+c*w[0]-r)])
```

```
def draw_line(a, b, color):
    pts = [np.array([a, b])]
    for _ in range(20):
        grad = nabla_branin(pts[-1])
        on_line = lambda t: branin(pts[-1]-grad*t)
        step = minimize_scalar(on_line, method='brent').x
        pts.append(pts[-1]-step*grad)
    plt.plot([pt[0] for pt in pts],
             [pt[1] for pt in pts],
              c = color)
```

Gradient Descent



Gradient Descent

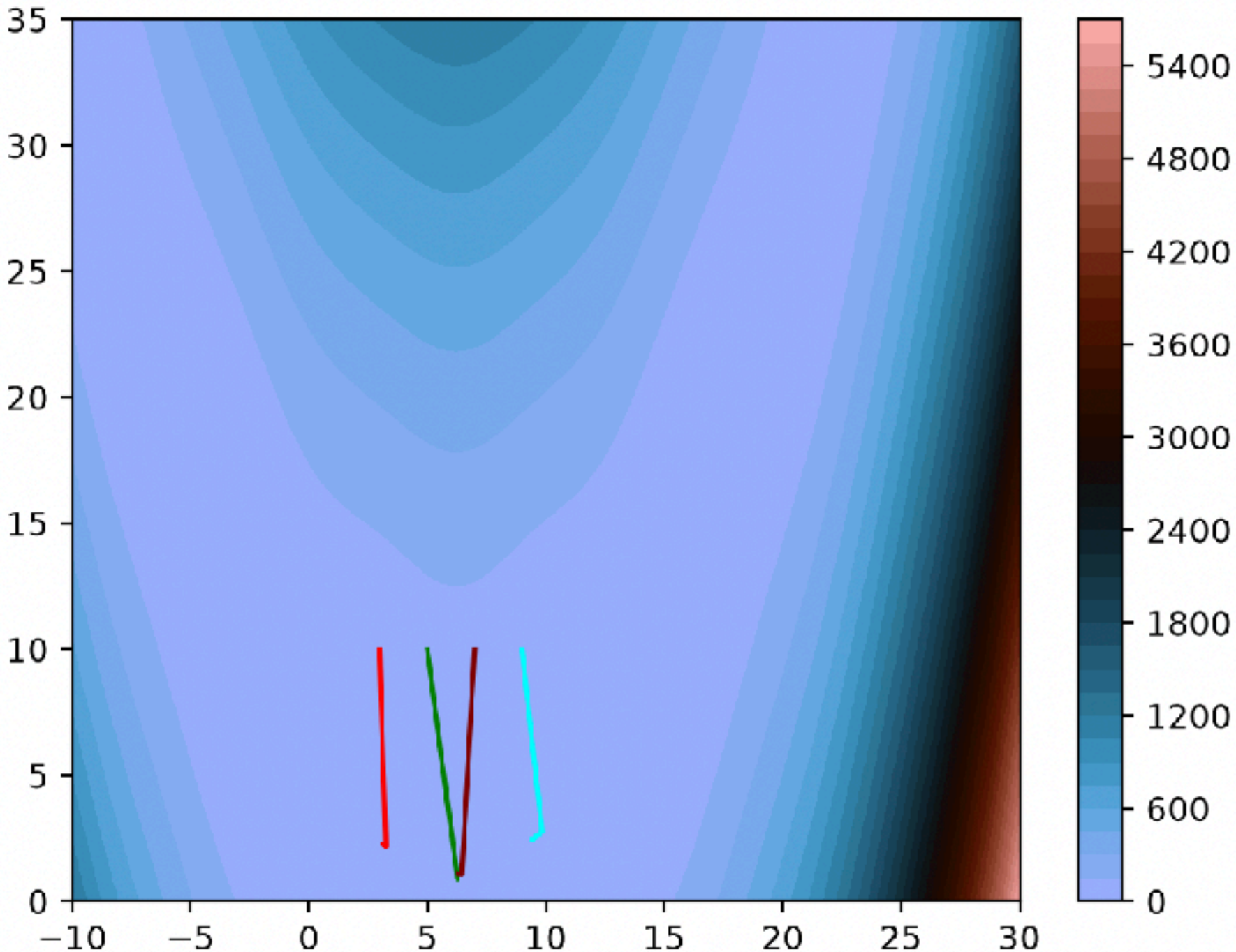
- Newton's method: Use the second derivative
  - $x_{k+1} = x_k - H_f(x_k) \cdot \nabla f(x_k)$
- This is basically a quadratic approximation

Gradient Descent

```
def branin_hessian(w):
    a, b, c = 1.0, 5.1 / (4 * np.pi**2), 5.0 / np.pi
    r, s, t = 6.0, 10.0, 1.0 / (8 * np.pi)
    return np.array([[2*a*(-2*b*w[0]+c)**2-4*a*b*
          (w[1]-b*w[0]**2+c*w[0]-r)-s*(1-t)*np.cos(w[0]),
           2*a*(-2*b*w[0]+c)],
         [2*a*(-2*b*w[0]+c), 2*a]])
```

```
def draw_newton(a, b, color):
    pts = [np.array([a, b])]
    for _ in range(20):
        grad = nabla_branin(pts[-1])
        hessian = branin_hessian(pts[-1])
        direction = -np.dot(np.linalg.inv(hessian), grad)
        pts.append(pts[-1] + direction)
    plt.plot([pt[0] for pt in pts],
             [pt[1] for pt in pts],
              c = color)
```

Gradient Descent



Gradient Descent

- Performance will depend on the function to be minimized
- 2nd derivative methods tend to have more problems, but if they work, they are faster
- Can get stuck in saddle points

Gradient Boosting

- Idea:
  - We use as loss function $\mathscr{L}(X) = (y - h(X))^2$
  - Then gradient is $\frac{\delta \mathscr{L}}{\delta h}(y, h(X)) = -(y - h(X))$
  - This is the negative of the residual $y - h(X)$

Gradient Boosting

- Algorithm:
  - Start with a single weak decision tree and train it
  - Iterate:
    - Get the predictions from the current ensemble `F`
    - Get the residuals from the current combined learner `y-F`
    - Fit a tree on the residuals
    - Get the predictions `hreg` for all data points
    - Find the α that minimizes the loss along the gradient, i.e. `np.linalg.norm(y-(F+a*hreg))**2`

Gradient Boosting

- Add α times the new tree to the ensemble

Gradient Boosting

```
def fit_gradient_boosting(X, y, n_estimators=250):
    n_samples, n_features = X.shape
    n_estimators = n_estimators
    estimators = []
    F = np.zeros((n_samples, ))
    for t in range(n_estimators):
    residuals = y - F
    h = DecisionTreeRegressor(max_depth=1)
    h.fit(X, residuals)
    hreg = h.predict(X)
    loss = lambda a: np.linalg.norm(
                          y - (F + a * hreg))**2
    a = minimize_scalar(loss, method='golden').x
    F += a * hreg
    estimators.append((a, h))
return estimators
```

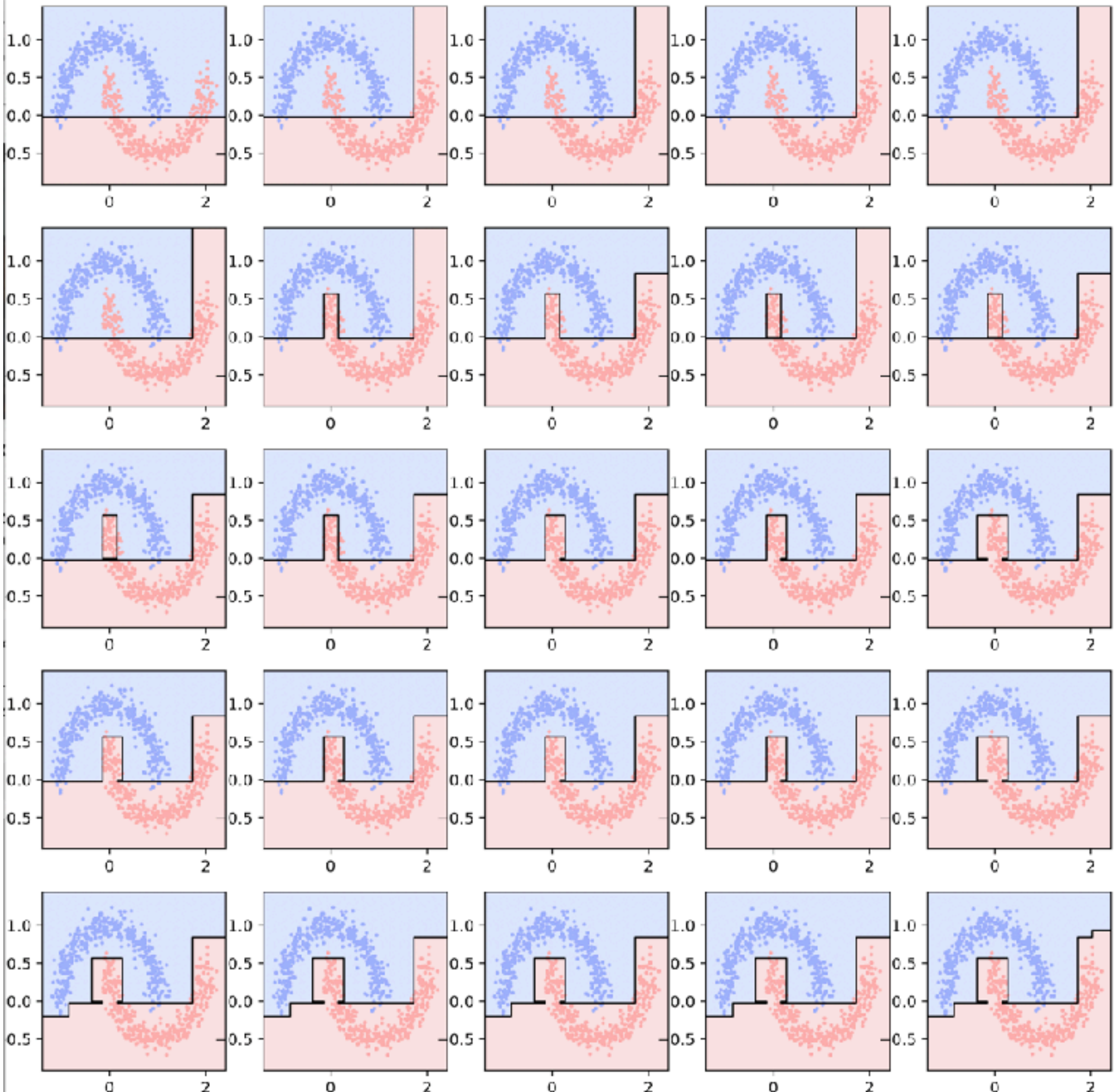
Gradient Boosting

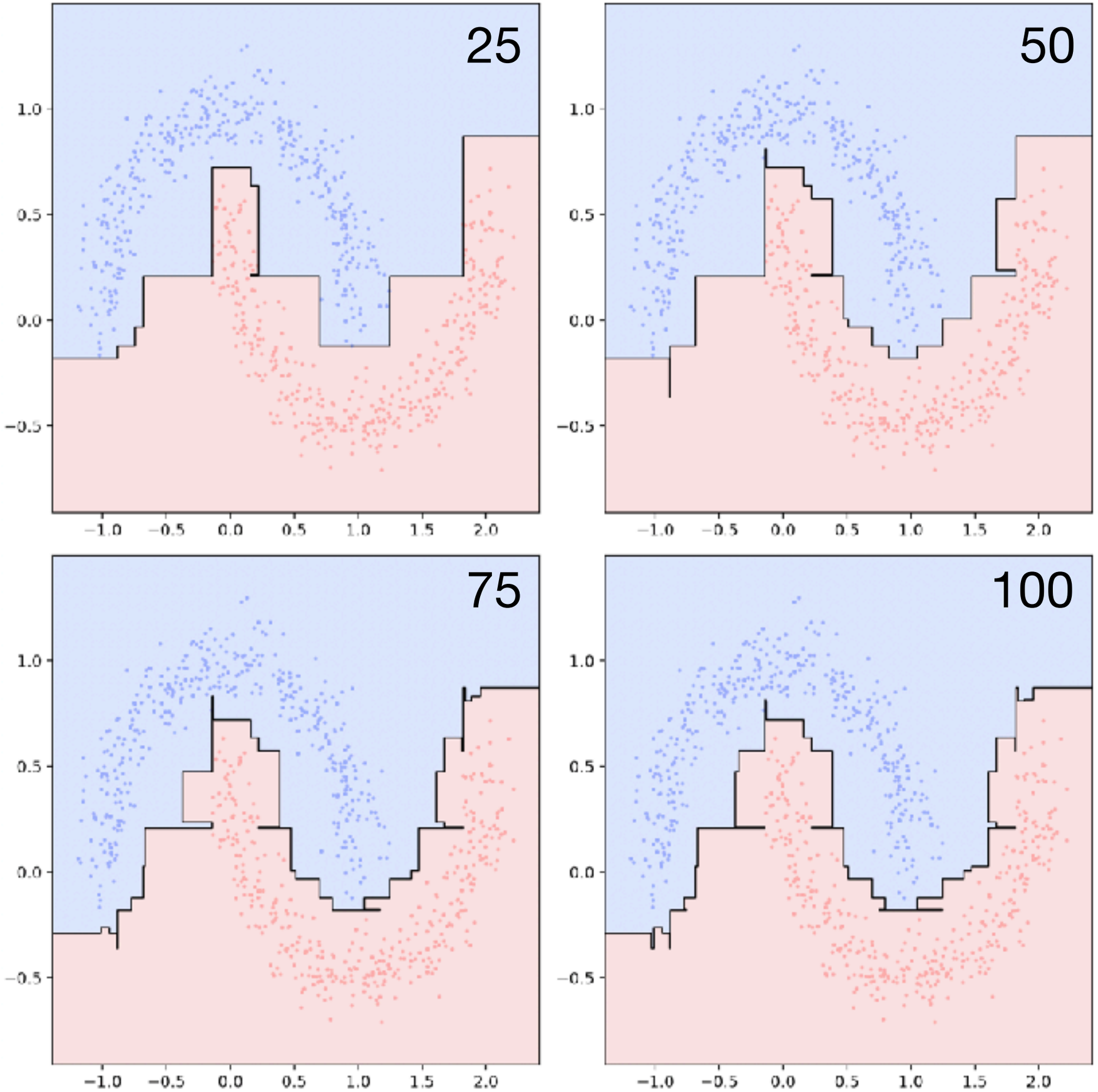
- The ensemble learner uses the alphas as a weight

```
def predict_gradient_boosting(X, estimators):
    pred = np.zeros((X.shape[0], ))
    for a, h in estimators:
        pred += a * h.predict(X)
    y = np.sign(pred)
    return y
```

Gradient Boosting

- Example:
  - Create moons
  - Show test set for iterations

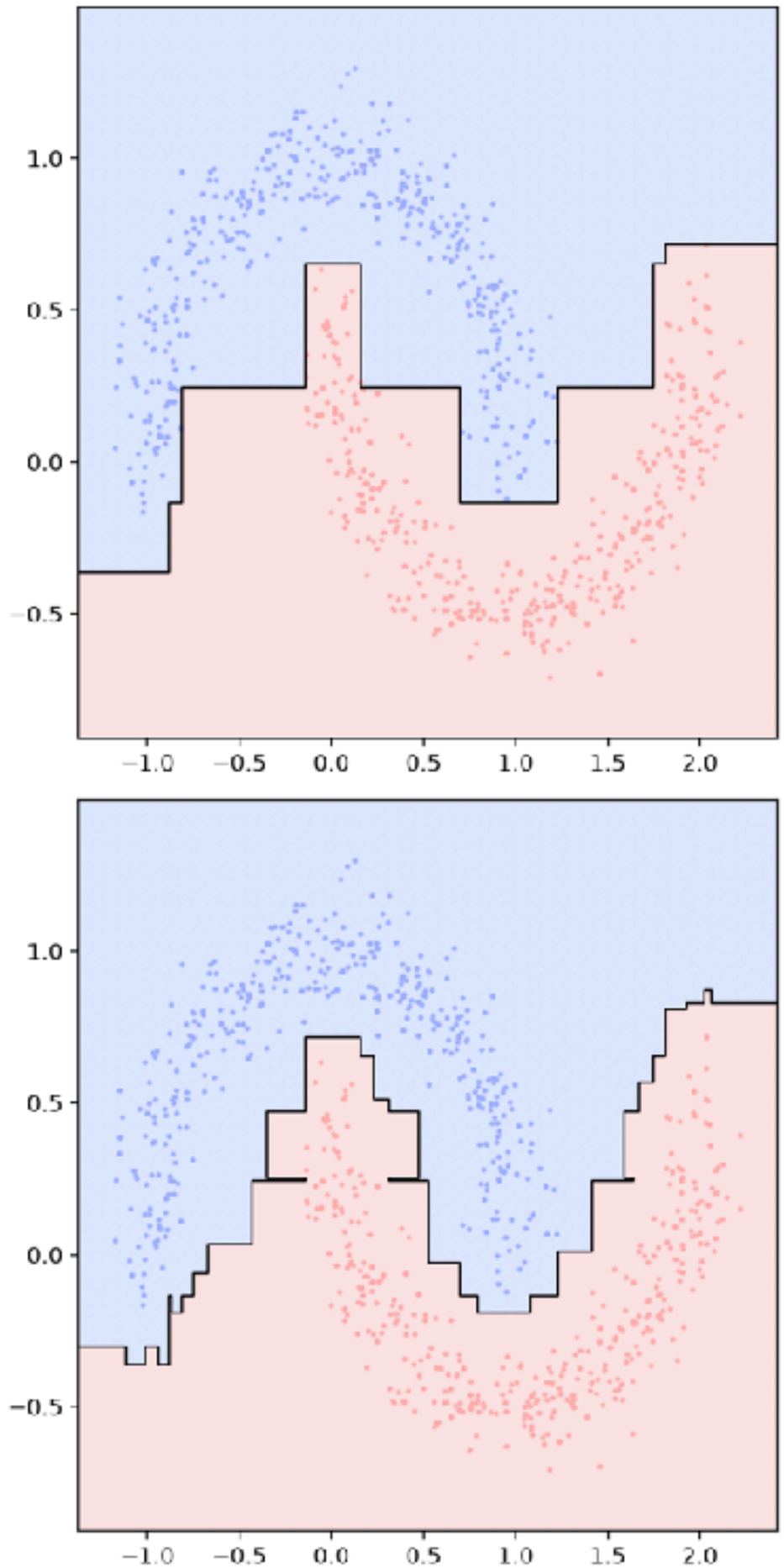

1.0
0.5
0.0
0.5
0
2

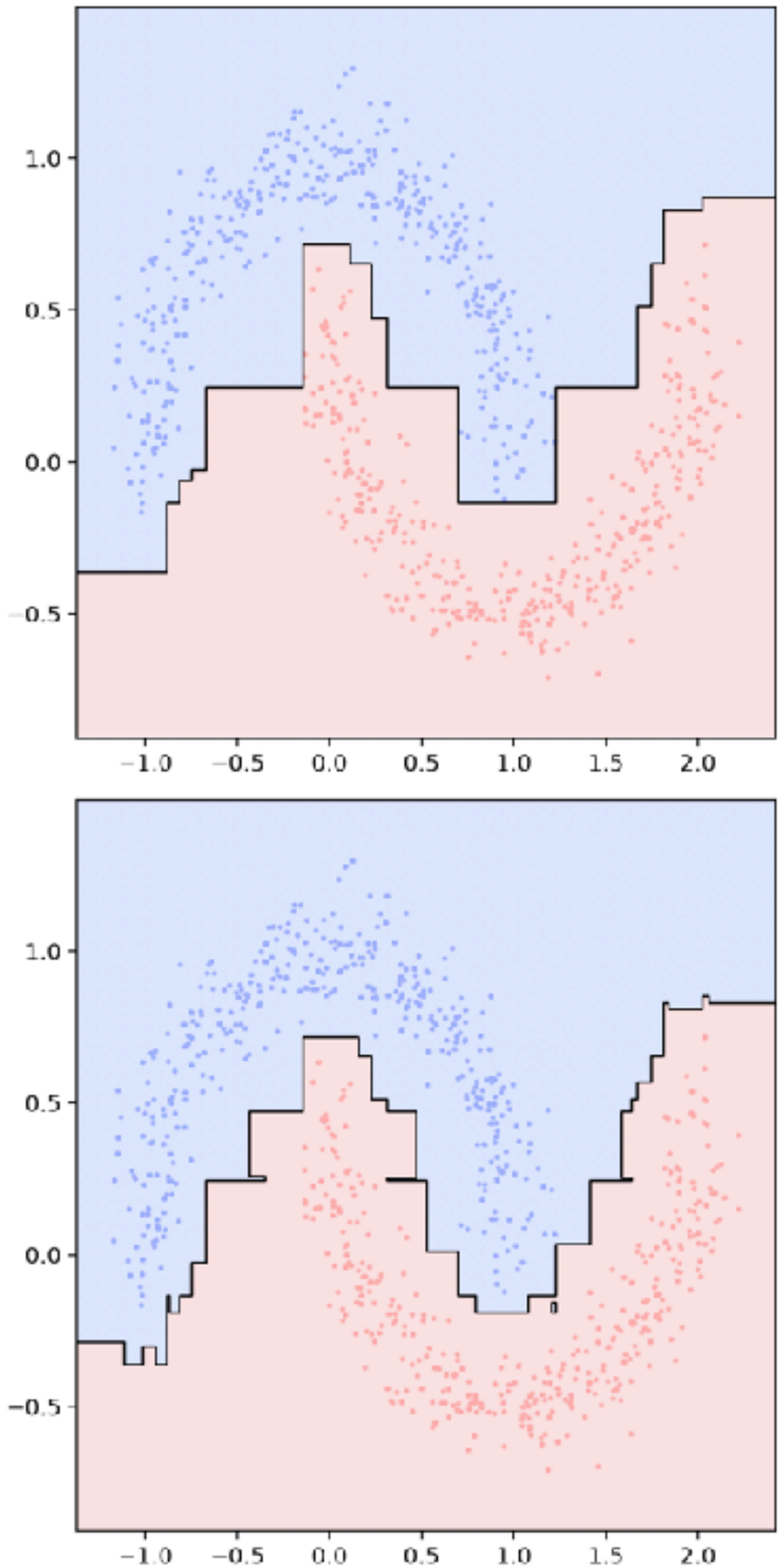

25
50
75
100
1.0
0.5
0.0
−0.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

Gradient Boosting

- Akin to lasso regularization, can use a damping factor

```
a = damping*minimize_scalar(loss, method='golden').x
```


1.0
0.5
0.0
−0.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


1.0
0.5
0.0
−0.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

1.0
0.5
0.0
−0.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

1.0
0.5
0.0
−0.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0



Newton Boosting

Newton Boosting

- Newton Boosting:
  - Use the gradient = residuals
  - Calculate weights of the data set using a Hessian

Newton Boosting

- Newton boosting approximates with a quadratic function
- If we use the square of differences as the loss function it is already quadratic
  - So, Newton boosting becomes gradient boosting
-

Newton Boosting

- The loss function is binary cross-entropy

$$\mathscr{L} = \sum_i - y_i \log \sigma(F_i) + (1 - y_i) \log(1 - \sigma(F_i))$$

- Since

$$\log \sigma(F_i) = \log(\frac{1}{1 + \exp(-F_i)}) = -\log(1 + \exp(-F_i))$$

$$\log(1 - \sigma(F_i)) = \log(1 - \frac{1}{1 + \exp(-F_i)}) = \log(1 + \exp(F_i))$$

- we have

$$\mathscr{L} = -\sum_i \left(y_i \cdot \log(1 + \exp(-F_i)) + (1 - y_i) \log(1 + \exp(F_i)) \right)$$

Newton Boosting

$$\mathscr{L} = \sum_i \left(y_i \cdot \log(1 + \exp(-F_i)) + (1 - y_i)\log(1 + \exp(F_i)) \right)$$

But $\dfrac{a+1}{\frac{1}{a}+1} = a$ implies $\log(1 + \exp(-F)) = \log(1 + \exp F) - F$

$$\mathscr{L} = \sum_i y_i(\log(1 + \exp(F_i) - F_i) + (1 - y_i)\log(1 + \exp(F_i))$$

$$\mathscr{L} = \sum_i \log(1 + \exp(F_i)) - y_i \cdot F_i$$

Newton Boosting

$$\mathscr{L} = \sum_i \log(1 + \exp(F_i)) - y_i \cdot F_i$$

- Gradient is $\frac{dL}{dF} = \sigma(F) - y$: this is the residual
- Hessian is $\sigma'(F) = \sigma(F)(1 - \sigma(F))$
  - Only a diagonal matrix

Newton Boosting

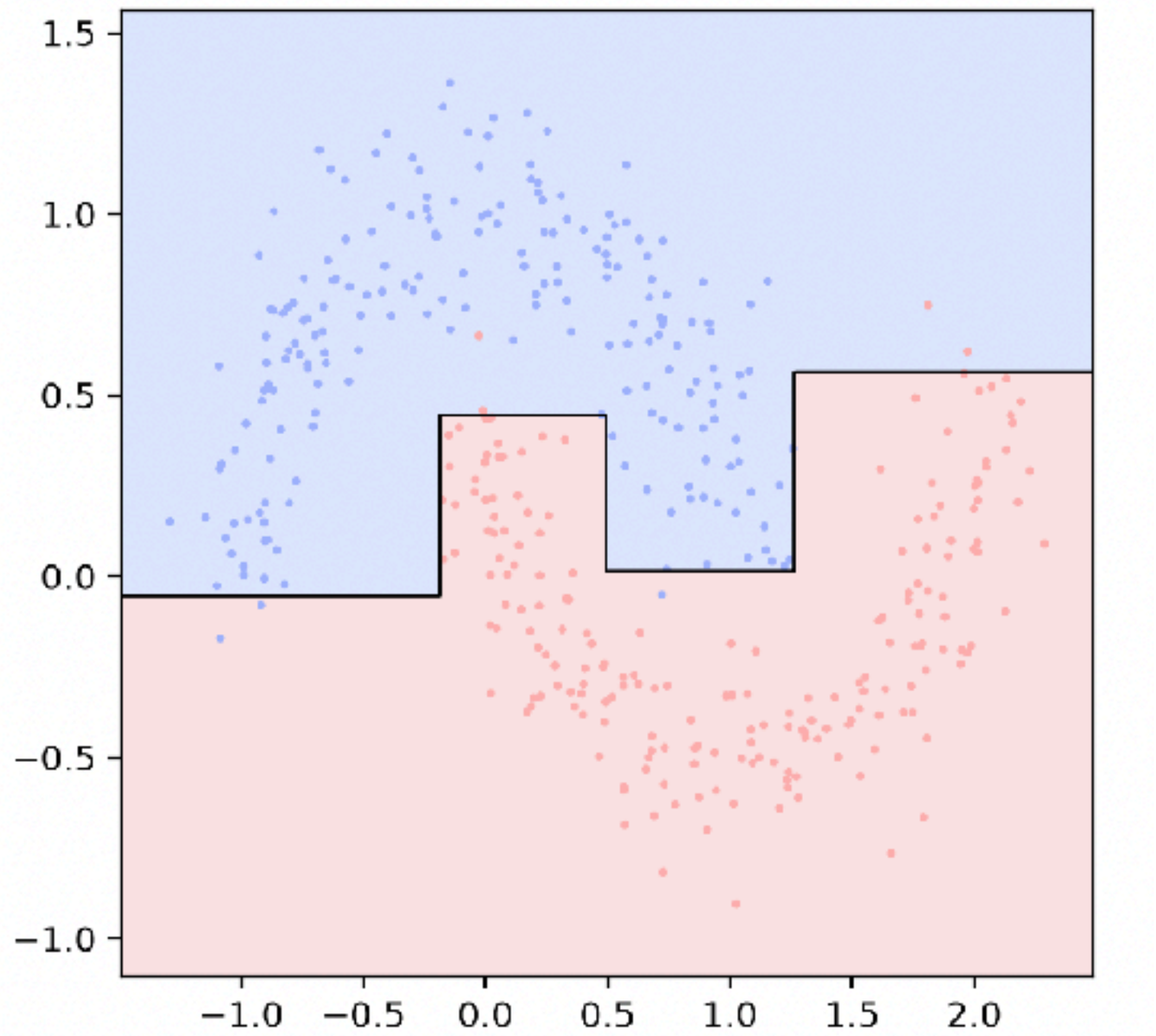
```
sigmoid = expit #Numerically superior

def logistic_loss(y, F):
    """Mean binary cross-entropy, via logaddexp so large |F|
       can't overflow."""
    return np.mean(np.logaddexp(0.0, F) - y * F)
```

```
def grad_hess(y, F):
    """First and second derivatives of the loss w.r.t. F, per
       sample.”""
    p = sigmoid(F)
    return p - y, p * (1.0 - p)
```

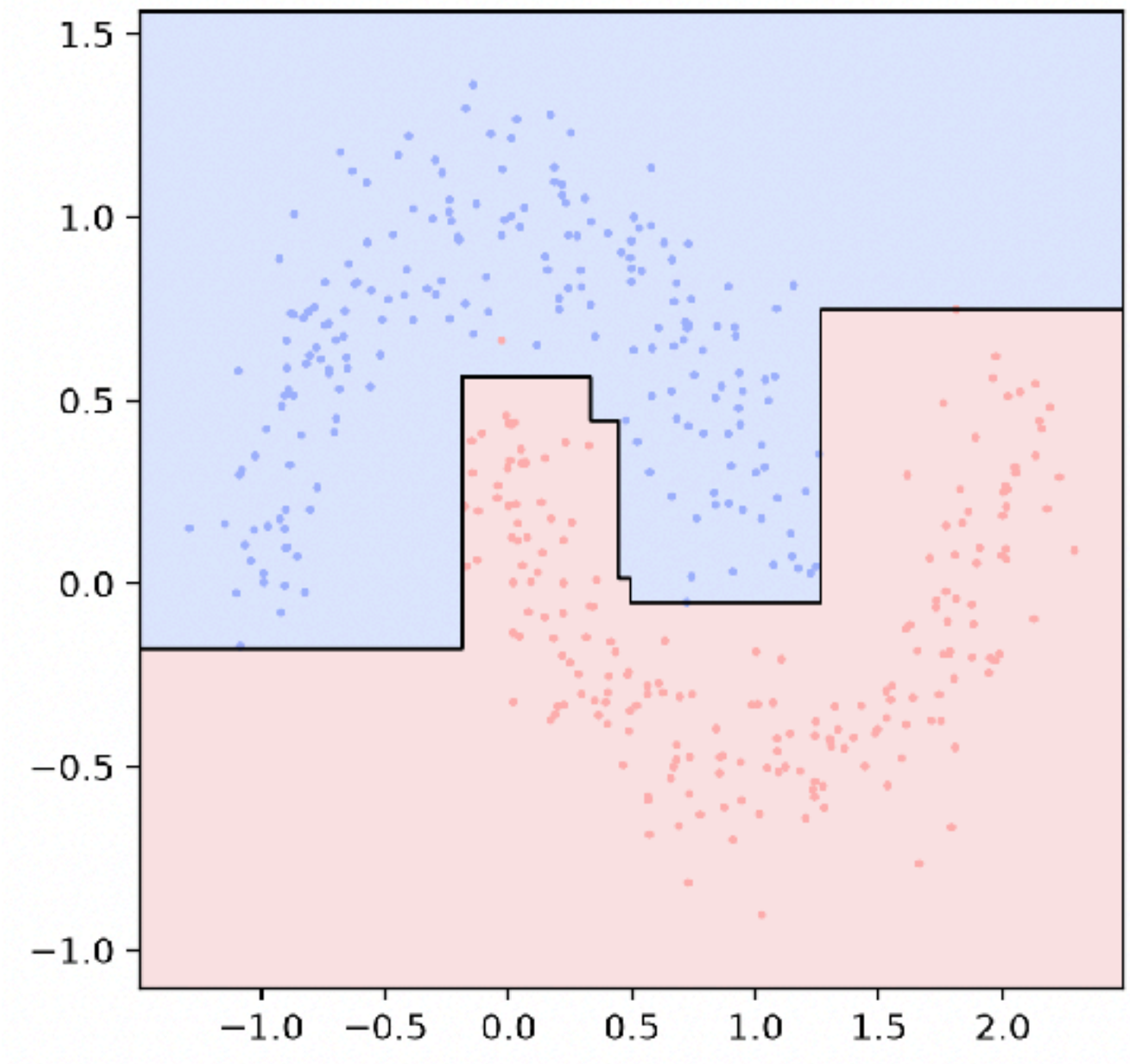
```
def fit(self, X, y):
    y = np.asarray(y, dtype=float)
    F = np.zeros(y.shape[0])
    self.trees = []
    for k in range(self.n_estimators):
        g, h = grad_hess(y, F)
        hc = np.maximum(h, self.min_hessian)
        target, weight = -g/hc, hc
        tree = DecisionTreeRegressor(
            max_depth=self.max_depth,
            random_state=self.random_state + k,
        )
        tree.fit(X, target, sample_weight=weight)
        self.trees.append(tree)
        F += self.learning_rate * tree.predict(X)
```

Newton Boosting



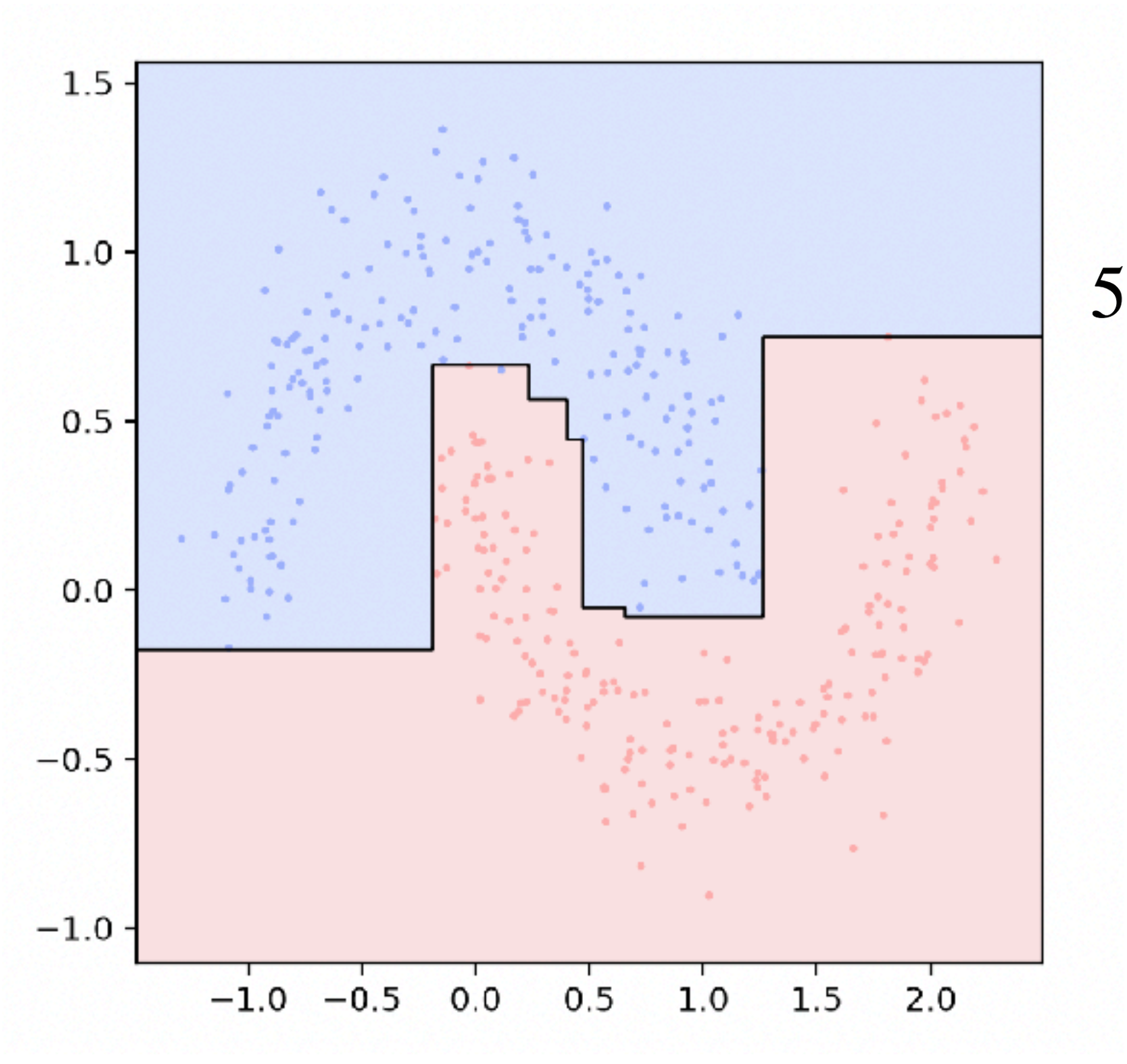
10 trees

Newton Boosting



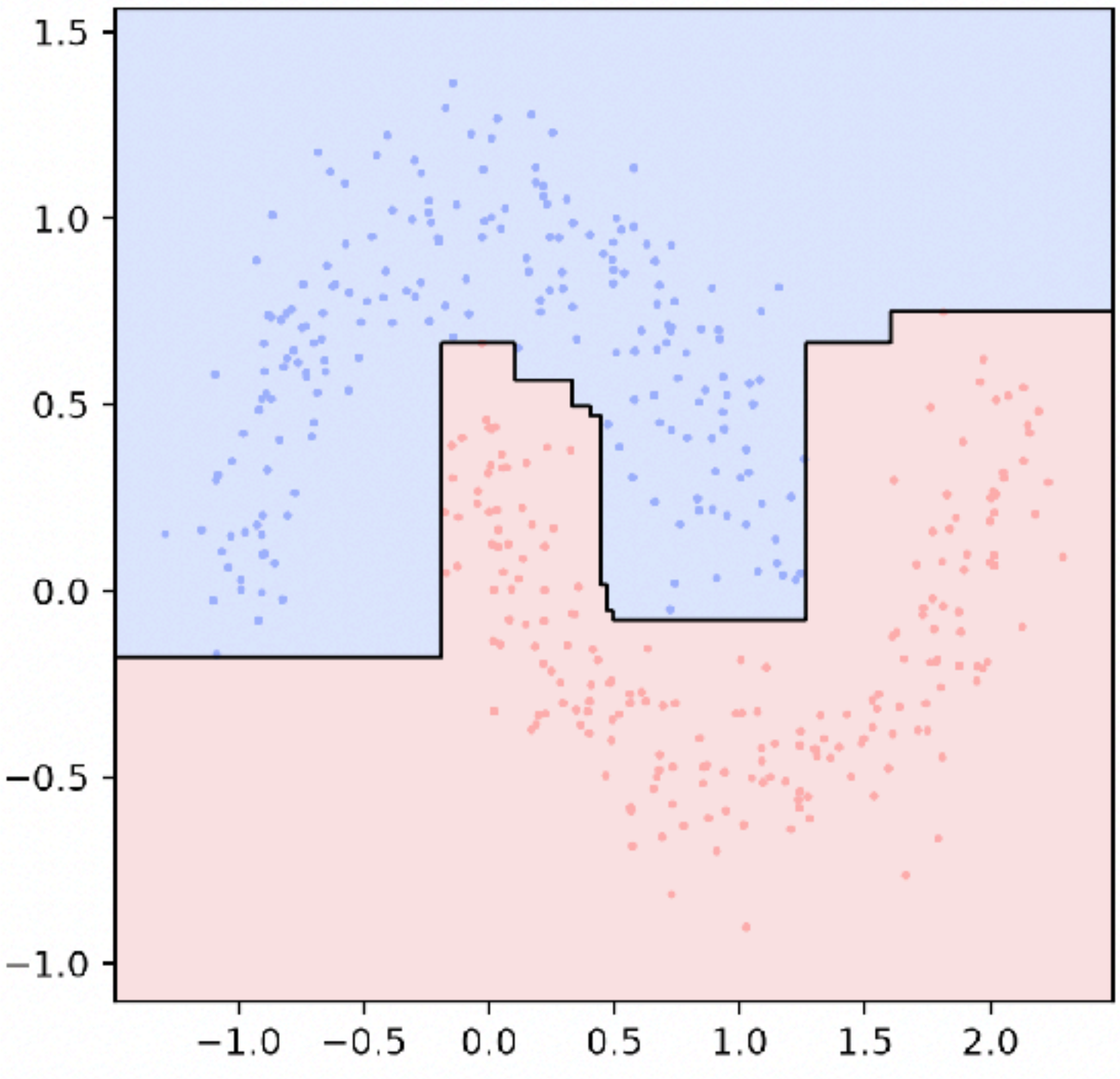
25 trees

Newton Boosting



50 trees

Newton Boosting



With 100 trees, learned the training set

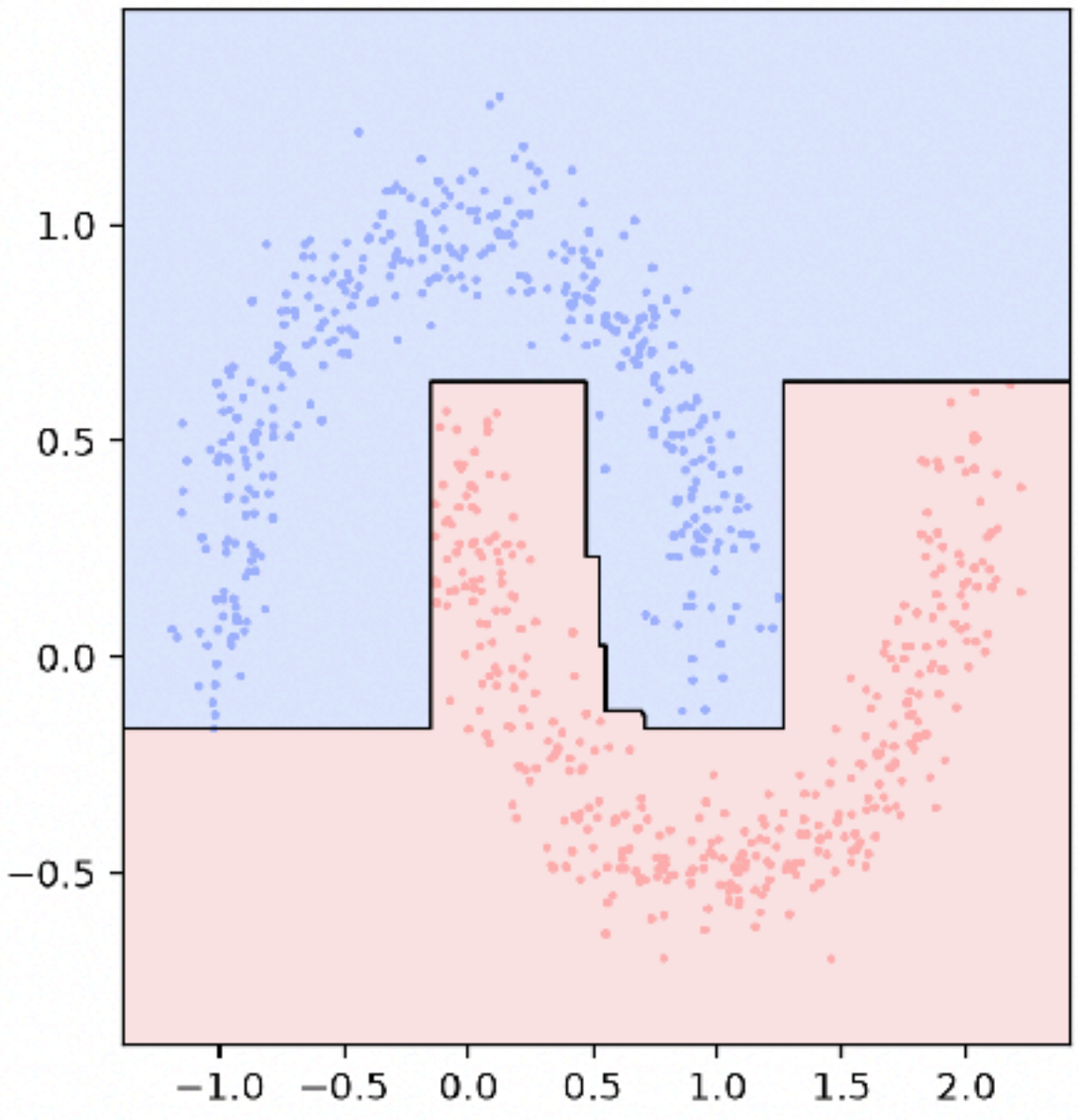
XGBoost

- XGBoost package (Tianqi Chen)
  - Regularization
  - Use histogram based search for best trees

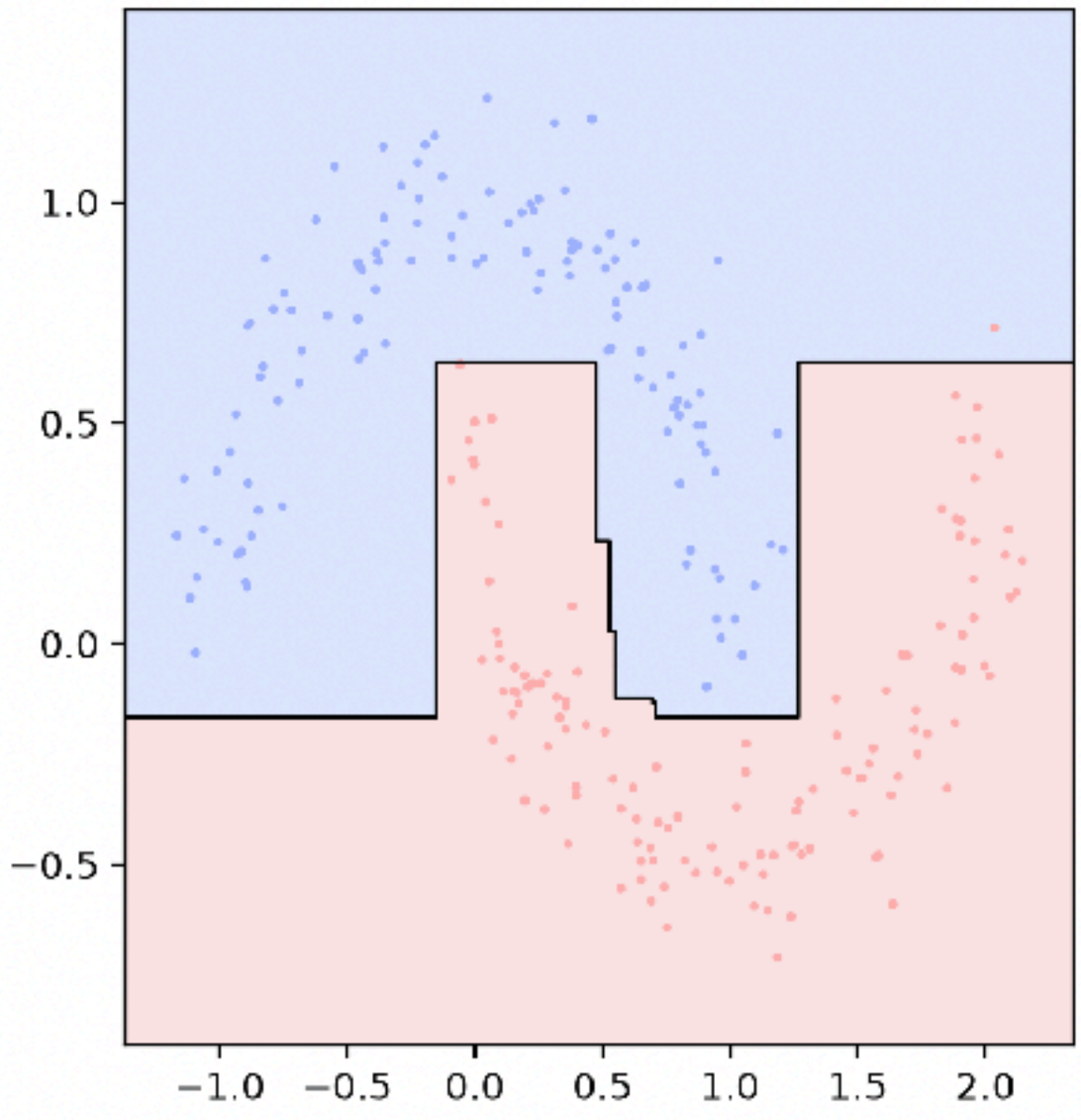
XGBoost

```
ens = XGBClassifier(n_estimators=100, max_depth=1,
                    objective='binary:logistic')
ens.fit(Xtrn, ytrn)
```

XGBoost



training set



test set